

FPTPost – Tài liệu thiết kế hệ thống (SPEC)

Phiên bản 1.0 • 08/10/2026 • Nhóm FPTPost • Đối tượng đọc: nhóm phát triển (6 thành viên)

1. Giới thiệu

1.1 Mục đích

Tài liệu này mô tả **cách xây dựng** FPTPost: kiến trúc, thiết kế dữ liệu, thuật toán và giao thức lỗi, tích hợp VNPay, bảo mật, frontend, DevOps, kiểm thử và quy ước làm việc. Câu hỏi 'làm gì' được trả lời bởi bộ đặc tả Excel; tài liệu này trả lời câu hỏi 'làm như thế nào'.

1.2 Quan hệ với bộ đặc tả Excel

Để tránh trùng lặp, SPEC **không sao chép** nội dung đã có trong tệp `FPTPost_Dac_ta_BR_UC.xlsx` mà chỉ tham chiếu tới nó.

Cần tra cứu	Nơi tra cứu
Yêu cầu nghiệp vụ, quy tắc, ràng buộc, mục tiêu (BR-F/N/R/C/G)	Sheet 03_BR
Use Case, luồng chính, ngoại lệ, người phụ trách, sprint	Sheet 04_UC, 05_Truy_vết
Mọi con số cấu hình (hạn mức, phí, thời hạn)	Sheet 06_Tham_số (mã PRM-xxx)
Công thức giá, hoa hồng và 20 ca kiểm thử công thức	Sheet 07
Danh mục trạng thái và bảng chuyển trạng thái	Sheet 08
Bảng/collection và các trường chính	Sheet 09
Danh mục endpoint	Sheet 10
Danh mục màn hình	Sheet 11
Đánh giá VNPay, phương án rút tiền	Sheet 12
Kế hoạch, phân bổ công việc, rủi ro	Sheet 13, 14, 15

Khi có mâu thuẫn: về **nghiệp vụ** (số liệu, quy tắc) thì Excel thắng; về **kỹ thuật** (cách làm) thì SPEC thắng. Mọi mâu thuẫn phải được ghi thành một issue gắn nhãn `spec-conflict`.

1.3 Quy ước

- Từ khóa **PHẢI, NÊN, CÓ THỂ** lần lượt chỉ mức bắt buộc, khuyến nghị, tùy chọn.
- PRM-xxx là tham số ở sheet 06; BR-x-nnn là yêu cầu ở sheet 03; UC-nn ở sheet 04; ADR-nn là quyết định kiến trúc ở mục 2.2; OQ-nn là câu hỏi còn mở ở Phụ lục A.
- Mã nguồn dùng tiếng Anh (lớp, biên, endpoint). Thông điệp hiển thị cho người dùng dùng tiếng Việt và được tách vào tệp tài nguyên.
- Thời gian lưu theo UTC, hiển thị theo UTC+7. Tiền là số nguyên VND, kiểu long trong Java và BIGINT trong SQL Server; cấm dùng số thực cho tiền.

2. Kiến trúc tổng thể

2.1 Nguyên tắc thiết kế

- Toàn vẹn tiền trước hiệu năng.** Mọi thao tác ảnh hưởng số dư đi qua một dịch vụ sổ cái duy nhất, chạy trong giao dịch cơ sở dữ liệu.
- Một nguồn sự thật cho trạng thái.** Trạng thái đơn, ví, ký quỹ nằm ở SQL Server; MongoDB chỉ chứa dữ liệu sự kiện có thể tái tạo hoặc chấp nhận mất một phần.
- Modular monolith.** Một khối triển khai, chia module có ranh giới rõ; không tách microservice trong phạm vi đồ án.
- API trước, mã sau.** Hợp đồng OpenAPI được thống nhất trước khi frontend và backend cùng phát triển.
- Idempotent theo mặc định cho thao tác ghi quan trọng.** Gửi lặp cùng yêu cầu không tạo kết quả thứ hai.
- Bất biến cho lịch sử.** Sổ cái, lịch sử trạng thái, nhật ký kiểm toán chỉ thêm, không sửa, không xóa.
- Cấu hình thay vì hằng số.** Mọi giá trị nghiệp vụ lấy từ bảng tham số (PRM), không viết cứng trong mã.
- Từ chối theo mặc định.** Endpoint mới mặc định yêu cầu xác thực và kiểm tra quyền; cho phép công khai phải được liệt kê tường minh.

2.2 Quyết định kiến trúc (ADR)

Mã	Quyết định	Lý do	Đánh đổi
ADR-01	Modular monolith trên Spring Boot 3 (JDK 21)	6 người, 12 tuần; giảm chi phí vận hành, giao dịch ACID xuyên module (đơn-ví)	Phải giữ kỷ luật ranh giới module, kiểm tra bằng ArchUnit

Mã	Quyết định	Lý do	Đánh đổi
ADR-02	SQL Server cho dữ liệu giao dịch; MongoDB cho dữ liệu sự kiện (vị trí, chat, thông báo, audit)	Dữ liệu tiền cần ACID và ràng buộc; dữ liệu sự kiện ghi nhiều, cần TTL và truy vấn địa lý; đúng nội dung đề cương	Hai kho dữ liệu: cần quản lý nhất quán ở mức ứng dụng
ADR-03	Sổ cái bút toán kép, chỉ thêm; số dư ví là bản chiếu có kiểm tra bất biến	Truy vết, đối soát, chống sai lệch	Mỗi giao dịch tiền ghi nhiều dòng; cần thiết kế chỉ mục tốt
ADR-04	JWT: Access Token ngắn hạn + Refresh Token xoay vòng lưu hash trong CSDL	Stateless cho API, thu hồi được phiên, phát hiện dùng lại token	Cần bảng phiên và logic xoay vòng
ADR-05	Thời gian thực bằng WebSocket giao thức STOMP trên Spring; có REST dự phòng	Chat và theo dõi hai chiều, tích hợp tốt với Spring	Cần xác thực khi bắt tay WebSocket và giới hạn kết nối
ADR-06	Rút tiền qua giao diện <code>PayoutProvider</code> (Mock, Manual, nhà cung cấp thật)	VNPay cổng thanh toán không có API chi tiền công khai (sheet 12)	Phải bảo trì nhiều bản triển khai
ADR-07	Outbox pattern cho sự kiện nội bộ phát sinh từ giao dịch DB	Không mất thông báo, không gửi khi giao dịch bị rollback	Thêm bảng outbox và worker
ADR-08	Tác vụ định kỳ bằng <code>@Scheduled</code> + ShedLock (khóa lưu trong SQL Server)	Chạy đúng một lần khi nhiều instance (BR-N Tác vụ định kỳ)	Phụ thuộc bảng khóa
ADR-09	Bản đồ: Leaflet + OpenStreetMap; định tuyến qua OSRM/OpenRouteService; dự phòng Haversine × 1,3	Miễn phí, đủ cho pilot (BR-C Phụ thuộc dịch vụ bản đồ)	Giới hạn lưu lượng, cần bộ nhớ đệm
ADR-10	Migration CSDL bằng Flyway	Tái lập trên mọi môi trường, chạy trong CI	Mọi thay đổi cấu trúc phải qua tệp migration

Mã	Quyết định	Lý do	Đánh đổi
ADR-11	Frontend lấy và lưu đệm dữ liệu bằng TanStack Query; trạng thái phiên bằng Context + Hooks	Khớp nội dung đề cương (Fetching and Caching Data, Context, Hooks); tránh Redux	Phải thống nhất quy ước khóa truy vấn
ADR-12	Lưu tệp ảnh trên hệ thống tệp/ đối tượng tương thích S3 (MinIO khi chạy Docker), mã hóa ảnh KYC ở tầng ứng dụng	Tách tệp khỏi CSDL, dễ thay bằng S3 thật	Cần quản lý khóa mã hóa

2.3 Bối cảnh hệ thống và giao tiếp bên ngoài

FPTPost gồm một SPA ReactJS, một backend Spring Boot và hai kho dữ liệu. Các hệ thống bên ngoài:

Hệ thống	Giao thức	Hướng	Mục đích	Cơ chế chịu lỗi
VNPay	HTTPS chuyển hướng (tạo thanh toán), HTTP GET IPN (VNPay → hệ thống), REST truy vấn và hoàn tiền	Hai chiều	Thu tiền, hoàn tiền	Timeout theo PRM-VNPAY_TIMEOUT, thử lại có giới hạn, circuit breaker, job truy vấn giao dịch treo
Dịch vụ định tuyến (OSRM/ORS)	HTTP REST	Ra	Quãng đường, thời gian, geocoding	Bộ nhớ đệm, dự phòng Haversine
Ô bản đồ OpenStreetMap	HTTPS (trình duyệt)	Trình duyệt → OSM	Hiển thị bản đồ	Tuân thủ chính sách sử dụng tile; có thể thay bằng nhà cung cấp khác
Email SMTP	SMTP	Ra	OTP, thông báo	Hàng đợi gửi, thử lại theo PRM-NOTIF_RETRY
SMS	HTTP (nhà cung cấp) hoặc Mock	Ra	OTP giao hàng, OTP đăng ký	Mock mặc định ở dev/demo; giao diện SmsGateway

Hệ thống	Giao thức	Hướng	Mục đích	Cơ chế chịu lỗi
Lưu trữ tệp	S3 API (MinIO)	Ra	Ảnh đơn, ảnh bằng chứng, ảnh KYC	Liên kết ký có thời hạn

2.4 Kiến trúc logic và module

Backend gồm các tầng: **API** (controller, DTO, xác thực) → **Application** (use case/service, giao dịch) → **Domain** (thực thể, quy tắc, máy trạng thái) → **Infrastructure** (JPA/Mongo repository, client ngoài).

Module backend	Phân hệ (sheet 03)	Trách nhiệm chính	Phụ thuộc cho phép
identity	AUTH, KYC	Tài khoản, phiên, KYC, hồ sơ, vai trò	common , audit
catalog	ADM (danh mục), SAF (từ khóa)	Loại hàng, khu vực, điểm hẹn, tham số, nội dung	common
order	ORD	Đơn, máy trạng thái đơn, lịch sử, phí hủy	identity , catalog , pricing , ledger
pricing	FEE	Tính giá, hoa hồng, khuyến mãi	catalog
market	MKT, TRP, REL	Sàn đơn, chào giá, chuyển đi, ghép đơn	order , identity , pricing
ledger	PAY	Sổ cái, ví, ký quỹ, hoàn tiền	common
payment	PAY	VNPay, PayoutProvider, rút tiền, đối soát	ledger , identity
tracking	TRK	Xác nhận lấy/giao, QR/OTP, vị trí	order , ledger (qua sự kiện)
reputation	REP, GAM	Đánh giá, điểm uy tín, thu nhập	order , identity
dispute	DSP, SAF	Khiếu nại, bồi thường, kiểm duyệt	order , ledger , identity
messaging	NEG, NTF	Chat, thông báo, WebSocket	identity (và lắng nghe sự kiện)

Module backend	Phân hệ (sheet 03)	Trách nhiệm chính	Phụ thuộc cho phép
admin	ADM	Dashboard, báo cáo, nhật ký, job	đọc từ các module
audit, common	Hệ thống	Nhật ký kiểm toán, lỗi chuẩn, tiện ích	không phụ thuộc module nghiệp vụ

Quy tắc phụ thuộc: không có phụ thuộc vòng; module chỉ gọi module khác qua giao diện
 *Facade /sự kiện miền, không truy cập repository của module khác. ArchUnit kiểm tra các quy tắc này trong CI.

Cấu trúc gói tham khảo:

```
com.fptpost
├─ common      (lỗi chuẩn, tiện ích, idempotency, thời gian, tiền)
├─ identity    (api, app, domain, infra)
├─ order       (api, app, domain, infra)
├─ ledger      (api, app, domain, infra)
├─ payment     (api, app, domain, infra, vn timer, payout)
└─ ...        (mỗi module cùng bốn tầng)
```

2.5 Công nghệ sử dụng

Phiên bản cụ thể được khóa trong `pom.xml` và `package-lock.json`; dùng bản ổn định mới nhất tương thích tại thời điểm khởi tạo.

Lớp	Công nghệ	Ghi chú
Ngôn ngữ/Runtime	Java 21 (JDK 21), Node 21	Theo đề cương SBA301
Backend	Spring Boot 3.x, Spring Web, Spring Validation, Spring Security 6, Spring Data JPA (Hibernate 6), Spring Data MongoDB, Spring WebSocket	Cốt lõi
CSDL	SQL Server (collation tiếng Việt), MongoDB	ADR-02
Hỗ trợ backend	Flyway, springdoc-openapi, MapStruct, Lombok, Resilience4j, ShedLock, thư viện JWT	ADR-04, 08, 10
Kiểm thử backend	JUnit 5, Mockito, Testcontainers, ArchUnit, JaCoCo, REST Assured	Mục 11

Lớp	Công nghệ	Ghi chú
Frontend	React 18, Vite, React Router 6, React-Bootstrap, TanStack Query, Axios, React Hook Form + Zod, React-Leaflet, @stomp/stompjs, Chart.js	Mục 9
Kiểm thử frontend	Vitest + Testing Library, Playwright hoặc Cypress	Mục 11
DevOps	Docker, Docker Compose, GitHub Actions, Nginx, k6	Mục 10

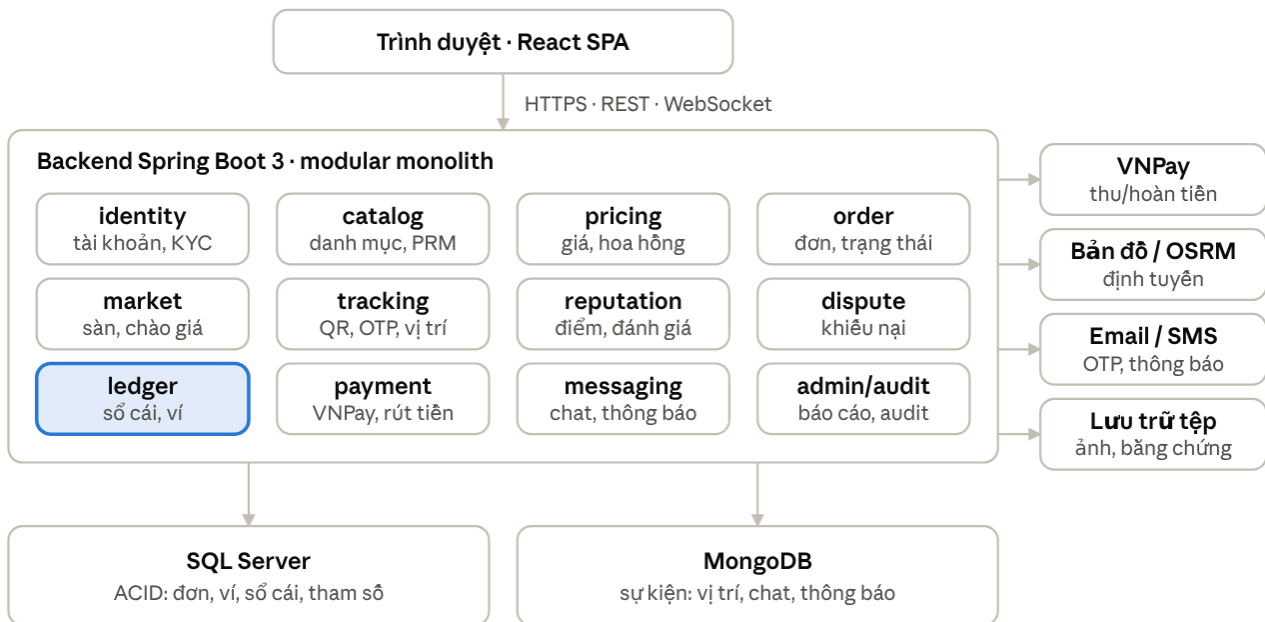
2.6 Môi trường và triển khai

Môi trường	Mục đích	Hạ tầng	Dữ liệu
local	Phát triển cá nhân	docker compose (SQL Server, MongoDB, MinIO, Mailpit), backend/frontend chạy IDE	Seed mẫu
dev	Tích hợp liên tục của nhánh chính	Docker Compose trên máy chủ nhóm, tunnel công khai cho IPN	Seed mẫu, reset hằng tuần
staging (demo)	Nghiệm thu, demo	Giống production: Nginx, backend 2 instance, TLS	Dữ liệu demo cố định
prod (tương lai)	Vận hành thật	Ngoài phạm vi đồ án	—

Cấu hình theo biến môi trường và profile Spring (`local` , `dev` , `staging` , `prod`); **cấm** đưa bí mật vào Git. Các biến bắt buộc: chuỗi kết nối CSDL, khóa ký JWT, khóa mã hóa dữ liệu, mã merchant và khóa bí mật VNPAY, thông tin SMTP/SMS, cấu hình lưu trữ tệp.

2.7 Sơ đồ kiến trúc

Một backend module hóa, hai kho dữ liệu, mọi dòng tiền đi qua sổ cái



Ồ nhân màu: module ledger là nơi duy nhất được ghi tiền.

kiến trúc logic · 12 module, 2 kho dữ liệu, 4 dịch vụ ngoài

Backend là một khối triển khai gồm các module nghiệp vụ; frontend chỉ nói chuyện với backend qua REST và WebSocket. Chỉ module **ledger** được phép ghi bút toán tiền, các module khác yêu cầu qua **LedgerFacade**. VNPay, dịch vụ bản đồ, email/SMS và lưu trữ tệp là các phụ thuộc ngoài, mỗi phụ thuộc đứng sau một giao diện để thay thế bằng bản Mock khi phát triển và kiểm thử.

3. Thiết kế dữ liệu

Danh sách thực thể và trường chính nằm ở sheet 09; mục này mô tả **quy ước, ràng buộc và cơ chế** mà sheet không diễn đạt được.

3.1 Phân kho dữ liệu và nhất quán giữa hai kho

Loại dữ liệu	Kho	Lý do
Tài khoản, đơn, chào giá, ví, sổ cái, ký quỹ, rút tiền, tham số	SQL Server	Cần ACID, khóa ngoại, ràng buộc duy nhất có điều kiện
Điểm vị trí, sự kiện hành trình	MongoDB	Ghi nhiều, cần TTL và chỉ mục địa lý 2dsphere
Chat, thông báo	MongoDB	Tài liệu lỏng, đọc theo phòng/người dùng, có TTL

Loại dữ liệu	Kho	Lý do
Nhật ký kiểm toán	MongoDB (chỉ thêm)	Khối lượng lớn; hành động tiền còn được ghi trong sổ cái SQL
Ảnh, tệp bằng chứng	Lưu trữ đối tượng	CSDL chỉ giữ khóa tệp và checksum

Nguyên tắc nhất quán: SQL Server là nguồn sự thật. MongoDB **không bao giờ** quyết định trạng thái nghiệp vụ; mất dữ liệu MongoDB không được làm sai sổ dư hay trạng thái đơn. Quy trình chuẩn: ghi SQL và bản ghi `outbox_events` trong cùng một giao dịch → worker đọc outbox, ghi MongoDB/gửi thông báo → đánh dấu đã xử lý. Worker **PHẢI** idempotent theo `eventId` (vì có thể chạy lại).

3.2 Quy ước SQL Server

Hạng mục	Quy ước
Tên	snake_case, bảng số nhiều (<code>orders</code>), cột khóa ngoại dạng <code>sender_id</code> , <code>order_id</code>
Khóa	<code>id</code> BIGINT IDENTITY dùng nội bộ; <code>public_id</code> UNIQUEIDENTIFIER (UUID) là mã duy nhất hiển thị ra API và MongoDB. Cấm lộ <code>id</code> tự tăng ra ngoài (chống đoán mã và IDOR). Mã vận đơn <code>tracking_code</code> là cột riêng theo định dạng ở sheet 03_BR
Tiền	BIGINT VND, có CHECK (≥ 0) ; không dùng DECIMAL/FLOAT cho tiền
Thời gian	DATETIME2(3) lưu UTC; Java dùng Instant ; chuyển UTC+7 ở tầng hiển thị
Enum	VARCHAR(30) + CHECK theo danh mục sheet 08; JPA <code>@Enumerated(EnumType.STRING)</code>
Tiếng Việt	NVARCHAR ; collation tiếng Việt (ví dụ <code>Vietnamese_100_CI_AS</code>). Tìm kiếm không dấu dùng cột chuẩn hóa <code>*_search</code> (bỏ dấu, chữ thường) có chỉ mục
Cột kiểm toán	<code>created_at</code> , <code>updated_at</code> , <code>created_by</code> , <code>updated_by</code> trên mọi bảng nghiệp vụ; bảng cập nhật đồng thời có <code>version</code> INT với <code>@Version</code>

Hạng mục	Quy ước
Xóa	Xóa mềm (<code>deleted_at</code>) chỉ cho hồ sơ, địa chỉ. Cấm xóa/sửa bảng sổ cái, lịch sử trạng thái, nhật ký
Migration	Flyway <code>V<yyyyMMddHHmm>__mo_ta.sql</code> ; không sửa migration đã merge; dữ liệu mẫu ở <code>R__seed_*.sql</code> chỉ chạy ở <code>local / dev</code>

3.3 Ràng buộc toàn vẹn then chốt

Ràng buộc	Cơ chế triển khai	Phục vụ
Một người giao cho một đơn, nhiều người nhận cùng lúc	<code>UPDATE orders SET courier_id=?, status='AWAITING_PAYMENT', version=version+1 WHERE id=? AND status='OPEN'</code> ; chỉ thành công nếu số dòng cập nhật bằng 1	BR-R Một người giao một đơn
CCCD duy nhất	Chỉ mục duy nhất có điều kiện trên <code>kyc_requests(id_number_hash)</code> WHERE <code>status='APPROVED'</code> ; kiểm tra thêm hồ sơ <code>PENDING</code> ở tầng ứng dụng	BR-R Duy nhất CCCD
Một yêu cầu rút đang xử lý mỗi người	Cột tính toán <code>is_open</code> BIT (1 nếu trạng thái thuộc <code>REQUESTED..PROCESSING</code>) + chỉ mục duy nhất <code>(user_id)</code> WHERE <code>is_open=1</code>	BR-R Giữ tiền khi chờ rút
Một chào giá hiệu lực mỗi người giao mỗi đơn	Chỉ mục duy nhất <code>(order_id, courier_id)</code> WHERE <code>status='PENDING'</code>	BR-R Một chào giá hiệu lực
Một đánh giá mỗi bên mỗi đơn	Duy nhất <code>(order_id, reviewer_id)</code>	BR-R Điều kiện đánh giá
Một khoản ký quỹ mỗi đơn	Duy nhất <code>escrows(order_id)</code>	BR-R Điều kiện ký quỹ
Không xử lý trùng giao dịch tiền	Duy nhất <code>ledger_transactions(idempotency_key)</code> , <code>payment_transactions(txn_ref)</code>	BR-N Giao dịch tiền idempotent
Số dư không âm	<code>wallets.available >= 0</code> (CHECK); công nợ truy thu ghi riêng ở <code>wallets.debt</code>	BR-R Công thức sổ dư ví

Công nợ: khi truy thu sau khiếu nại vượt số dư khả dụng, phần thiếu ghi vào `wallets.debt` (không để `available` âm); các khoản giải ngân sau này **PHẢI** cần trừ nợ trước.

3.4 Sổ cái bút toán kép

3.4.1 Cấu trúc

`ledger_transactions` (một dòng mỗi nghiệp vụ tiền) có nhiều `ledger_entries` (mỗi dòng một tài khoản và chỉ một trong hai cột `debit`, `credit`; CHECK (`debit >= 0 AND credit >= 0 AND (debit = 0 OR credit = 0)`)). Quy tắc:

- `LedgerService.post(txn)` là **điểm ghi duy nhất**; kiểm tra tổng nợ bằng tổng có, ghi toàn bộ dòng và cập nhật `wallets` trong **cùng một giao dịch DB**.
- Tài khoản DB của ứng dụng bị DENY UPDATE, DELETE trên hai bảng này; sửa sai bằng bút toán điều chỉnh có phê duyệt.
- Cập nhật số dư chống đồng thời bằng câu lệnh điều kiện: `UPDATE wallets SET available = available + :delta, version = version + 1 WHERE user_id=:u AND available + :delta >= 0`; số dòng cập nhật = 0 thì từ chối (không đủ tiền hoặc xung đột).
- Khóa idempotency theo mẫu: `topup:{txnRef}`, `escrow-hold:{orderId}`, `release:{orderId}`, `refund:{orderId}:{n}`, `wd-request:{id}`, `wd-paid:{id}`, `wd-reverse:{id}`. Gọi lại với cùng khóa **trả về giao dịch cũ**, không ghi thêm.
- Thứ tự khóa khi một giao dịch đựng nhiều ví: luôn khóa theo `user_id` tăng dần để tránh deadlock.

3.4.2 Hệ thống tài khoản

Mã tài khoản	Loại	Ý nghĩa
<code>USER_AVAILABLE:{userId}</code>	Nợ phải trả người dùng	Số dư khả dụng của người dùng
<code>USER_PENDING_WD:{userId}</code>	Nợ phải trả	Tiền đang chờ rút
<code>USER_DEBT:{userId}</code>	Tài sản	Công nợ người dùng nợ nền tảng (truy thu thiếu)
<code>ESCROW (gắn order_id)</code>	Nợ phải trả	Tiền đang ký quỹ theo đơn
<code>VNPAY_CLEARING</code>	Tài sản	Tiền đã thu qua VNPay, chờ đối soát/thanh toán về tài khoản merchant

Mã tài khoản	Loại	Ý nghĩa
PAYOUT_CLEARING	Tài sản	Tiền đã chi cho người dùng qua PayoutProvider
PLATFORM_REVENUE	Doanh thu	Hoa hồng (sau trích quỹ), phí rút
GUARANTEE_FUND	Nợ phải trả (quỹ)	Quỹ bảo đảm (trích hoa hồng + phí bảo hiểm)
PROMO_EXPENSE	Chi phí	Khuyến mãi, thưởng do nền tảng chịu
TAX_WITHHELD	Nợ phải trả	Thuế khấu trừ chờ nộp (nếu cấu hình)
SUSPENSE	Trung gian	Chênh lệch đối soát chờ xử lý

3.4.3 Quy tắc ghi sổ (posting rules)

Ký hiệu: Nợ = ghi nợ (Dr), Có = ghi có (Cr). Giá trị tiền tính theo công thức sheet 07: $agr = \text{phí giao thông nhất}$, $ins = \text{phí bảo hiểm}$, $promo = \text{khuyến mãi}$, $pay = agr + ins - promo$, $comm = \text{hoa hồng}$, $fc = \text{phần trích quỹ}$, $wh = \text{thuế khấu trừ}$, $net = agr - comm - wh$.

Nghịệp vụ	Nợ	Có
Nạp tiền thành công (IPN)	VNPAY_CLEARING (số tiền)	USER_AVAILABLE: người nạp
Ký quỹ bằng ví	USER_AVAILABLE: người gửi (pay); PROMO_EXPENSE (promo)	ESCROW (agr + ins)
Ký quỹ bằng VNPay trực tiếp (IPN)	VNPAY_CLEARING (pay); PROMO_EXPENSE (promo)	ESCROW (agr + ins)
Giải ngân	ESCROW (agr + ins)	USER_AVAILABLE: người giao (net); TAX_WITHHELD (wh); PLATFORM_REVENUE (comm - fc); GUARANTEE_FUND (fc + ins)
Hoàn tiền toàn phần vào ví	ESCROW (agr + ins)	USER_AVAILABLE: người gửi (pay); PROMO_EXPENSE (promo, đảo khuyến mãi)
Hủy có phí hủy cf	ESCROW (agr + ins)	USER_AVAILABLE: người giao (cf); USER_AVAILABLE: người gửi (pay - cf); PROMO_EXPENSE (promo, đảo)

Nghệp vụ	Nợ	Có
Hoàn về nguồn VNPay (sau khi đã hoàn vào ví)	USER_AVAILABLE: người gửi	VNPAY_CLEARING
Tạo yêu cầu rút	USER_AVAILABLE (amount)	USER_PENDING_WD (amount)
Rút thành công, phí wf	USER_PENDING_WD (amount)	PAYOUT_CLEARING (amount - wf); PLATFORM_REVENUE (wf)
Rút bị từ chối/thất bại/hủy	USER_PENDING_WD (amount)	USER_AVAILABLE (amount)
Bồi thường từ Quỹ	GUARANTEE_FUND	USER_AVAILABLE: người được bồi thường
Truy thu người giao sau khiếu nại	USER_AVAILABLE: người giao (phần có sẵn); USER_DEBT (phần thiếu)	USER_AVAILABLE: người gửi (toàn bộ)
Thưởng, giới thiệu	PROMO_EXPENSE	USER_AVAILABLE
Điều chỉnh đối soát	SUSPENSE hoặc tài khoản liên quan	tài khoản liên quan hoặc SUSPENSE (có phê duyệt, ghi audit)

Bất biến cần kiểm tra (job `LedgerIntegrityJob` và API `/admin/ledger/integrity`): (1) với mọi giao dịch, tổng nợ = tổng có; (2) `wallets.available`, `escrow_held`, `pending_withdraw` khớp số dư tính từ bút toán; (3) tổng số dư `ESCROW` bằng tổng `escrows` ở trạng thái `HELD/RELEASE_PENDING/FROZEN`; (4) không có `USER_AVAILABLE` âm. Vi phạm bất kỳ bất biến nào là sự cố mức cao: đóng bằng giao dịch tiền mới và cảnh báo Admin.

3.5 MongoDB

Mọi tham chiếu đến đơn/người dùng dùng `public_id` (chuỗi UUID), không dùng khóa tự tăng của SQL Server.

Collection	Ví dụ tài liệu rút gọn	Chỉ mục	Vòng đời
location_pings	{orderId, courierId, loc: {type: 'Point', coordinates: [lng, lat]}, speed, accuracy, seq, at}	{orderId:1, at:-1}, 2dsphere(loc), TTL at	TTL = PRM-TRACK_TTL
tracking_events	{orderId, type, actorId, at, payload, loc}	{orderId:1, at:1}	Giữ theo đơn
chat_rooms	{orderId, participants: [userId], status, createdAt}	duy nhất {orderId:1}	Đóng chỉ-đọc sau PRM-DISPUTE_WINDOW
chat_messages	`{roomId, senderId, type:'TEXT`	IMAGE	SYSTEM', text, mediaKey, flags: {masked}, readBy:[], at}`
notifications	{userId, type, title, body, data:{orderId, deepLink}, channels, readAt, dedupeKey, at}	{userId:1, at:-1}, duy nhất {userId:1, dedupeKey:1}	Giữ 6 tháng, dọn bằng job
audit_logs	{actor: {id, role, ip}, action, entity: {type, id}, before, after, correlationId, at}	{entity.type:1, entity.id:1, at:-1}, {actor.id:1, at:-1}	Giữ PRM-AUDIT_KEEP; người dùng DB chỉ có quyền insert

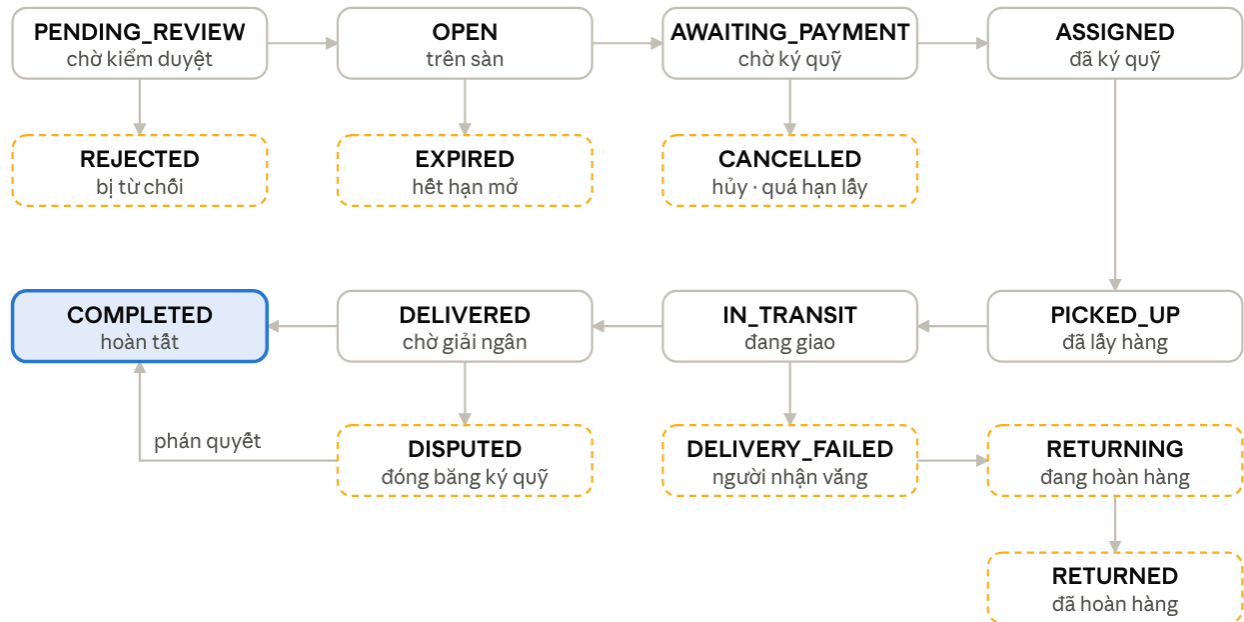
Lưu ý triển khai: (1) dedupeKey của thông báo = eventId + kênh để outbox chạy lại không tạo thông báo trùng; (2) audit_logs không dùng TTL index vì thời hạn 5 năm, dọn bằng job có kiểm soát; (3) đọc vị trí gần nhất dùng `find({orderId}).sort({at:-1}).limit(1)` và được cache ngắn ở bộ nhớ ứng dụng.

4. Thiết kế nghiệp vụ lỗi

4.1 Máy trạng thái của đơn

Trạng thái và bảng chuyển trạng thái đầy đủ nằm ở sheet 08. Mục này quy định **cách triển khai** để không thể đổi trạng thái sai.

Vòng đời đơn: 8 trạng thái chính và 7 nhánh ngoại lệ



Nét đứt: nhánh ngoại lệ. Ô nhân màu: kết quả của luồng giao thành công.

vòng đời đơn · 8 trạng thái chính, 7 nhánh ngoại lệ

Hình chỉ vẽ nhánh chính và các nhánh kết thúc. Hai vòng quay về **OPEN** (hết giữ chỗ ký quỹ; người giao hủy sau khi ký quỹ) và đường **COMPLETED** → **DISPUTED** (khiếu nại sau giải ngân) xem ở sheet 08.

Mọi thay đổi trạng thái **PHẢI** đi qua `OrderStateMachine.transition(order, event, actor, ctx)`; không controller hay service nào được gán `status` trực tiếp. Các bước trong một lần chuyển, tất cả trong **một giao dịch DB**:

1. Nạp đơn và khóa lạc quan bằng `version`; nếu trạng thái hiện tại khác trạng thái kỳ vọng thì trả `409 ORDER_STATE_CONFLICT`.
2. Kiểm tra quyền của tác nhân (vai trò và quyền sở hữu) theo ma trận phân quyền sheet 02.
3. Kiểm tra điều kiện (guard) của sự kiện, ví dụ hạng và hạn mức người giao, thời hạn ký quỹ, OTP đúng.
4. Đổi trạng thái và thêm một dòng vào `order_status_history` (từ, đến, người thực hiện, lý do).

5. Thực hiện tác động tiền qua `LedgerFacade` (ký quỹ, giải ngân, hoàn tiền); lỗi ở bước này làm hủy toàn bộ giao dịch.
6. Ghi `outbox_events` loại `OrderStatusChanged` để phát thông báo, timeline MongoDB và WebSocket sau khi commit.

Cặp (trạng thái, sự kiện) không có trong bảng chuyển trả 409

`ILLEGAL_STATE_TRANSITION`. Kiểm thử đơn vị **PHẢI** duyệt toàn bộ tổ hợp trạng thái × sự kiện và khẳng định đúng tập chuyển hợp lệ ở sheet 08.

4.2 Thuật toán giá và hoa hồng

Công thức, tham số và 20 ca kiểm thử nằm ở sheet 07 và 06. Phần này quy định cách tính trong mã để kết quả **khớp từng đồng** với bảng tính.

- **Chỉ dùng số nguyên.** Hệ số khẩn cấp và các tỷ lệ lưu dạng điểm cơ bản (bps): $\times 1,0 = 10000$, $\times 1,2 = 12000$, $\times 1,5 = 15000$; hoa hồng 8% = 800. Quãng đường tính bằng mét (`long`).
- **Chỉ làm tròn một lần ở cuối.** Cộng mọi thành phần theo đơn vị milli-đồng (một phần nghìn đồng) rồi mới nhân hệ số và làm tròn lên bội số của `PRM-ROUND_FEE`. Cách này tránh sai lệch do số thực; bảng tính xử lý bằng `ROUND(x, 2)` trước `CEILING`.
- **Khóa giá.** Đơn lưu `price_suggested`, `agreed_price`, `fee_rate_locked` và phiên bản tham số tại thời điểm tạo hoặc chấp nhận; đổi tham số sau đó không ảnh hưởng đơn đang chạy (BR-R Áp dụng thay đổi tham số).
- **Khoảng cách.** `RoutingClient` trả mét theo đường bộ, có bộ nhớ đệm theo tọa độ làm tròn 5 chữ số thập phân; nếu lỗi thì dùng Haversine $\times 1,3$ và gán cờ `approximate=true` để giao diện hiển thị ước tính gần đúng.

```

long suggest(Quote q, Params p) {
    long wFee = q.kg <= 2 ? 0 : q.kg <= 5 ? p.w2 : q.kg <= 10 ? p.w3 : p.w4;
    long sFee = q.longestCm <= p.sizeS ? 0 : q.longestCm <= p.sizeM ?
p.sizeFeeM : p.sizeFeeL;
    long baseM = p.baseKm * 1000L, longM = p.longKm * 1000L, m = q.meters;
    long milli = (p.baseFare + wFee + sFee) * 1000L
                + p.perKm * Math.min(Math.max(0, m - baseM), longM - baseM)
                + p.perKmLong * Math.max(0, m - longM);
    long scaled = milli * urgencyBps(q.urgency, p); // nhân 10_000
    long step = p.roundFee * 1000L * 10_000L;
    return Math.max(p.minFare, Math.floorDiv(scaled + step - 1, step) *
p.roundFee);
}

long commission(long agr, int rateBps) { return Math.floorDiv(agr * rateBps,
10_000); }
long fundShare(long comm, int fundBps) { return Math.floorDiv(comm * fundBps,
10_000); }
// net = agr - commission - withholding; phần lẻ do làm tròn thuộc về người
giao

```

Bắt buộc: 20 ca ở sheet 07 được xuất ra `src/test/resources/pricing-cases.csv` và chạy trong kiểm thử đơn vị; một ca lệch là build đỏ.

4.3 Thuật toán ghép đơn-chuyển

Mục tiêu: trả tối đa `PRM-MATCH_TOPN` đơn phù hợp cho một chuyển (và ngược lại, người giao phù hợp cho một đơn) trong ≤ 2 giây với 5.000 đơn mở (BR-N Hiệu năng gợi ý ghép đơn).

- Lọc thô bằng SQL.** Lấy đơn `OPEN` có khối lượng $\leq$ sức chứa còn lại và điểm lấy nằm trong hộp bao (bounding box) của chuyển, mở rộng thêm `PRM-MATCH_DETOUR`. Dùng chỉ mục (`status`, `pickup_lat`, `pickup_lng`).
- Rút gọn tuyến.** Đơn giản hóa polyline của chuyển bằng Douglas-Peucker, dung sai 50 m, còn tối đa 200 điểm; lưu sẵn khi đăng chuyển.
- Tính độ lệch.** Với điểm lấy A và điểm giao B của đơn, tính khoảng cách ngắn nhất tới polyline bằng xấp xỉ equirectangular (đủ chính xác trong đô thị): $x = \Delta\lambda \cdot \cos(\varphi_{tb})$, $y = \Delta\varphi$, $d = 6371 \cdot \sqrt{x^2 + y^2}$ km, chiều lên từng đoạn. Yêu cầu $\max(dA, dB) \leq \text{PRM-MATCH_DETOUR}$.
- Đúng chiều.** Vị trí chiều của A dọc tuyến phải đứng **trước** vị trí chiều của B.
- Khớp thời gian.** Thời điểm xe tới điểm chiều của A, ước tính bằng `depart_at` + quãng đường / vận tốc trung bình của phương tiện, phải nằm trong khung giờ lấy hàng của đơn.

6. Chấm điểm và xếp hạng theo công thức dưới đây, lấy MATCH_TOPN điểm cao nhất.

```
score = 0.50 * (1 - detour / MATCH_DETOUR)    // càng ít lệch càng cao
        + 0.20 * timeFit                      // 1 nếu sớm hơn hạn, giảm
tuyến tính khi sát hạn
        + 0.20 * (courierTrustScore / 100)
        + 0.10 * priceFit                      // giá niêm yết so với giá sàn
mong muốn của chuyên
```

Trọng số là tham số kỹ thuật đặt trong `application.yml` (không phải số nghiệp vụ nên không nằm ở sheet 06); nếu nhóm muốn Admin chỉnh thì bổ sung thành PRM. Chiều ngược lại (người giao phù hợp cho một đơn) dùng cùng hàm chấm điểm trên các chuyên `SCHEDULED` và `ACTIVE`.

Kiểm thử: bộ dữ liệu cố định gồm một tuyến mẫu và 30 đơn (đúng chiều, ngược chiều, lệch 2 km, lệch 5 km, quá cân, sai giờ) với kết quả kỳ vọng; thêm benchmark 5.000 đơn.

4.4 Điểm uy tín và hạng

Tham số và ngưỡng hạng ở sheet 06; đây là công thức tính. Điểm nằm trong $[0, 100]$, mỗi thành phần nằm trong $[0, 1]$:

```
score = 100 * ( W_ONTIME      * onTime
                + W_RATING     * (ratingAvg / 5)
                + W_COMPLETE   * completion
                + W_COMPLAINT  * (1 - complaintLoss) )
```

Trong đó `onTime` = đơn giao đúng hạn / đơn hoàn tất; `completion` = đơn hoàn tất / đơn đã nhận (không tính đơn bị hủy do người gửi); `complaintLoss` = khiếu nại bị xử thua / đơn hoàn tất (tối đa 1). Bốn trọng số cộng lại bằng 1.

Làm mượt cho người mới. Mỗi thành phần dùng trung bình Bayes $x' = (n \cdot x + K \cdot 0,5) / (n + K)$ với $K = 5$ (hằng kỹ thuật), để người giao chưa có đơn nào có điểm đúng bằng `PRM-TRUST_INIT` (50) và không dao động mạnh ở các đơn đầu.

Cập nhật. Cập nhật gia tăng khi đơn hoàn tất, khi nhận đánh giá và khi khiếu nại có phán quyết; tính lại toàn bộ hằng đêm bằng `TrustRecomputeJob`. Hạng là hạng cao nhất mà cả điểm lẫn số đơn đều đạt ngưỡng; xuống hạng ngay ở lần tính lại nếu không còn đạt. Mỗi lần đổi điểm hoặc hạng ghi một dòng `trust_score_history` kèm các thành phần để giải thích được biến động. Hạn mức nhận đơn và hoa hồng đọc từ tham số theo hạng hiện tại; hoa hồng của đơn đã nhận **không đổi** theo BR-R Tính hoa hồng khi giải ngân.

4.5 Giao thức xác nhận bằng QR và OTP

Có bốn mục đích xác nhận (`purpose`), cùng dùng một cơ chế:

purpose	Hợp lệ khi đơn ở	Bên sinh mã	Bên quét/nhập
PICKUP	ASSIGNED	Người gửi (màn hình QR lấy hàng)	Người giao
DELIVERY	IN_TRANSIT	Người nhận (trang theo dõi) hoặc OTP gửi SMS	Người giao
HANDOVER	Chặng đang chờ bàn giao	Người giao chặng trước	Người giao chặng sau
RETURN	RETURNING	Người gửi	Người giao

Mã QR. Nội dung `fp1.<orderPublicId>.<purpose>.<expEpoch>.<nonce>.<sig>` với `sig = Base64Url(HMAC-SHA256(K_qr, orderPublicId|purpose|exp|nonce))`. Khóa `K_qr` lưu ngoài mã nguồn, có định danh `kid` để xoay khóa. Mã sống `PRM-QR_TTL` và được làm mới khi hết hạn; `handover_tokens` chỉ lưu hash của `nonce`, `used_at` và `attempts`.

Các bước kiểm tra khi quét (đúng thứ tự):

1. Tách mã, kiểm tra chữ ký bằng so sánh thời gian hằng định (constant-time).
2. Kiểm tra chưa hết hạn.
3. Kiểm tra mã chưa dùng.
4. Kiểm tra `purpose` đúng với trạng thái đơn (bảng trên).
5. Kiểm tra người gọi là người giao của đơn (hoặc người gửi với mục đích `RETURN`).
6. Đánh dấu đã dùng bằng `UPDATE ... SET used_at=now WHERE used_at IS NULL` (chỉ một yêu cầu thắng).
7. Gọi `OrderStateMachine.transition`.

Sai ở bước nào thì tăng `attempts`; vượt `PRM-DELIVERY_OTP_FAIL` thì khóa nhập trong `PRM-DELIVERY_OTP_LOCK` và hướng dẫn liên hệ Hỗ trợ.

OTP giao hàng. 6 chữ số sinh bằng `SecureRandom`; lưu `HMAC-SHA256(K_otp, orderId|otp)`, không lưu rõ và không ghi log; gửi SMS cho người nhận khi đơn sang `IN_TRANSIT`; gửi lại tối đa theo `PRM-OTP_RESEND`; hết hiệu lực khi đơn kết thúc.

Giao ngoại lệ. Bắt buộc có ít nhất một ảnh, lý do và tọa độ; đơn gắn cờ `exception_delivery` và ký quỹ chờ `PRM-AUTO_RELEASE` (BR-R Giao ngoại lệ không có OTP).

Cảnh báo mềm. Lưu khoảng cách giữa tọa độ lúc xác nhận và điểm giao; nếu lớn hơn 300 m (hằng kỹ thuật) thì gắn cờ `geo_mismatch` cho Kiểm duyệt xem xét khi có khiếu nại, không chặn thao tác.

4.6 Tác vụ định kỳ

Tất cả chạy bằng @Scheduled + ShedLock (ADR-08), cron theo UTC, ghi kết quả vào job_runs , lỗi thì cảnh báo Admin ở màn hình SCR-61. Mỗi job **PHẢI** idempotent: chạy lại hoặc chạy chồng không tạo kết quả thứ hai.

Job	Lịch	Việc	Idempotent nhờ
ExpireOpenOrders	1 phút	Đơn OPEN quá hạn mở → EXPIRED	Điều kiện status trong UPDATE
ExpireOffers	1 phút	Chào giá PENDING quá PRM-BID_TTL → EXPIRED	Điều kiện status
ReleasePaymentHold	1 phút	AWAITING_PAYMENT quá PRM-PAY_HOLD → OPEN, gỡ người giao	Điều kiện status
PickupOverdue	5 phút	ASSIGNED quá giờ hẹn + PRM-PICKUP_GRACE → CANCELLED, hoàn tiền, trừ điểm	Khóa refund: {orderId}:{n}
AutoRelease	1 phút	Ký quỹ RELEASE_PENDING đến hạn và không FROZEN → giải ngân	Khóa release: {orderId}
VnpayPendingQuery	2 phút	Giao dịch PENDING quá 2 phút → gọi truy vấn VNPay, chốt trạng thái	Khóa topup: {txnRef}
OutboxDispatcher	2 giây	Gửi sự kiện outbox theo lô 100	eventId
Reconciliation	PRM-RECONCILE_AT	Đối soát giao dịch ngày trước	Khóa theo ngày đối soát
LedgerIntegrity	1 giờ	Kiểm tra bất biến sổ cái (mục 3.4.3)	Chỉ đọc
TrustRecompute	03:00	Tính lại điểm và hạng	Ghi đè theo người giao
CleanupCredentials	03:30	Xóa OTP và refresh token hết hạn	Điều kiện thời gian
RetentionJob	Hằng tuần	Dọn thông báo, nhật ký kiểm toán quá hạn lưu	Điều kiện thời gian

5. Tích hợp VNPay

Đánh giá mức phù hợp và phương án rút tiền ở sheet 12; mục này là thiết kế tích hợp. Chi tiết tham số và định dạng **luôn lấy theo tài liệu sandbox hiện hành của VNPay**; nếu khác với những gì nêu ở đây thì tài liệu của VNPay thắng và nhóm cập nhật SPEC.

5.1 Phạm vi và thành phần

FPTPost dùng năm chức năng của cổng thanh toán: tạo thanh toán (chuyển hướng), Return URL, IPN, truy vấn giao dịch và hoàn tiền. **Không** dùng chức năng chi tiền (xem ADR-06).

Thành phần	Vai trò
<code>PaymentGateway</code> (giao diện)	Ranh giới để thay bằng Mock khi kiểm thử
<code>VnpayClient</code>	Dùng liên kết thanh toán, ký và kiểm chữ ký, gọi truy vấn và hoàn tiền
<code>VnpayProperties</code>	Cấu hình từ biến môi trường: mã merchant, khóa bí mật, địa chỉ cổng, Return URL, thời gian chờ
<code>VnpayIpnController</code>	Nhận IPN, gọi <code>PaymentResultHandler</code>
<code>VnpayReturnController</code>	Nhận Return URL, xác minh chữ ký chỉ để hiển thị, chuyển người dùng về <code>/payment/result</code>
<code>PaymentResultHandler</code>	Một điểm xử lý kết quả dùng chung cho IPN và truy vấn, idempotent

Địa chỉ IPN được đăng ký trên cổng quản trị merchant của VNPay và **PHẢI** truy cập được từ Internet (dev dùng tunnel; staging dùng tên miền thật).

5.2 Tạo yêu cầu thanh toán

Bộ tham số theo tài liệu công khai (phiên bản 2.1.0): `vnp_Version`, `vnp_Command=pay`, `vnp_TmnCode`, `vnp_Amount`, `vnp_CurrCode=VND`, `vnp_CreateDate`, `vnp_ExpireDate`, `vnp_IpAddr`, `vnp_Locale=vn`, `vnp_OrderInfo`, `vnp_OrderType`, `vnp_ReturnUrl`, `vnp_TxnRef`, `vnp_SecureHash`.

Quy tắc	Nội dung
Số tiền	<code>vnp_Amount = số tiền VND × 100</code> , số nguyên, không dấu phân cách

Quy tắc	Nội dung
Mã tham chiếu	vnp_TxnRef duy nhất, định dạng <mã mục đích> <yyMMddHHmmss><6 ký tự ngẫu nhiên> (ví dụ TP261009103045A7K2QX); khớp payment_transactions.txn_ref
Thời gian	vnp_CreateDate và vnp_ExpireDate định dạng yyyyMMddHHmmss theo UTC+7; hết hạn = tạo + PRM-VNPAY_EXPIRE
Nội dung	vnp_OrderInfo tiếng Việt không dấu, không ký tự đặc biệt, ví dụ Nap vi FPTPost TP261009103045A7K2QX
Địa chỉ IP	Lấy từ X-Forwarded-For chỉ khi yêu cầu đến từ proxy tin cậy (Nginx của hệ thống)
Chữ ký	Sắp tham số theo khóa tăng dần, nối khóa=giá trị mã hóa URL bằng & (bỏ vnp_SecureHash), ký HMAC-SHA512 bằng khóa bí mật; so sánh chữ ký không phân biệt hoa thường và bằng so sánh thời gian hằng định
Thời gian tạo	Tạo liên kết và trả cho frontend trong ≤ 3 giây (BR-N Tạo thanh toán VNPay)

5.3 Luồng nạp tiền

Bước	Bên	Hành động
1	Frontend	POST /wallet/topups với số tiền và header Idempotency-Key
2	Backend	Kiểm tra PRM-MIN_TOPUP , PRM-MAX_TOPUP , PRM-MAX_BALANCE ; tạo payment_transactions trạng thái CREATED ; dựng liên kết; chuyển PENDING ; trả paymentUrl , txnRef , expiresAt
3	Frontend	Chuyển trình duyệt sang paymentUrl
4	Người dùng	Thanh toán trên trang VNPay
5	VNPay	Gọi IPN tới backend (server-to-server)
6	Backend	PaymentResultHandler xử lý (mục 5.4), phản hồi cho VNPay

Bước	Bên	Hành động
7	VNPay	Chuyển trình duyệt về Return URL của backend
8	Backend	Xác minh chữ ký chỉ để hiển thị, chuyển về <code>/payment/result?txnRef=...</code>
9	Frontend	Gọi trạng thái giao dịch từ CSDL; nếu còn <code>PENDING</code> thì thăm dò mỗi 3 giây tối đa 30 giây rồi hiển thị đang xử lý

Trang kết quả **không bao giờ** tin tham số trên URL: trạng thái hiển thị luôn lấy từ bản ghi đã được IPN hoặc truy vấn xác nhận (BR-R Chỉ ghi có khi VNPay xác nhận).

5.4 Xử lý IPN

`PaymentResultHandler.apply(result, source)` là điểm xử lý duy nhất cho kết quả từ IPN và từ truy vấn (`source = IPN | QUERY`); Return URL **không** gọi hàm này. Các bước, theo thứ tự:

1. Xác minh chữ ký; sai thì phản hồi `RspCode=97` .
2. Tìm giao dịch theo `vnp_TxnRef` ; không thấy thì `RspCode=01` .
3. So `vnp_Amount / 100` với số tiền đã lưu; lệch thì đánh dấu `MISMATCH` , cảnh báo Admin, phản hồi `RspCode=04` và **không** ghi sổ cái.
4. Nếu giao dịch đã ở trạng thái cuối (`SUCCESS` , `FAILED`) thì phản hồi `RspCode=02` (đã xác nhận trước đó), không xử lý lại.
5. Trong **một giao dịch DB**: khóa dòng giao dịch (`UPDLOCK`); nếu `vnp_ResponseCode = 00` và `vnp_TransactionStatus = 00` thì chuyển `SUCCESS` và ghi sổ cái (`TOPUP` , hoặc `ESCROW_HOLD` với mục đích `ORDER` , kèm chuyển đơn `AWAITING_PAYMENT` → `ASSIGNED`); ngược lại chuyển `FAILED` . Lưu nguyên văn tham số nhận được để đối soát.
6. Phản hồi `RspCode=00` trong ≤ 2 giây; thông báo và việc nặng đẩy qua outbox.

Ba trường hợp biên **PHẢI** xử lý:

- **Đơn không còn chờ ký quỹ** (người gửi hủy hoặc hết giữ chỗ trước khi IPN đến): ghi có vào ví người gửi như một lần nạp tiền và thông báo; tiền không bao giờ bị mất.
- **IPN thành công nhưng giao dịch đã EXPIRED** (đánh dấu hết hạn trước khi IPN đến): chuyển `MISMATCH` để đối soát; Admin hoặc hoàn về nguồn qua VNPay, hoặc ghi có thủ công bằng bút toán điều chỉnh có phê duyệt.
- **IPN đến trước hoặc sau Return URL**: kết quả như nhau vì chỉ IPN và truy vấn được ghi nhận.

Mã phản hồi IPN theo mẫu của VNPay: 00 xác nhận thành công, 01 không thấy đơn, 02 đơn đã xác nhận, 04 sai số tiền, 97 sai chữ ký, 99 lỗi khác.

5.5 Truy vấn và giao dịch treo

Job `VnpayPendingQuery` gọi API truy vấn cho mọi giao dịch `PENDING` quá 2 phút và chuyển kết quả cho `PaymentResultHandler` với `source = QUERY`. Với API truy vấn và hoàn tiền, chuỗi ký là các trường **nội bằng dấu | theo đúng thứ tự tài liệu VNPay quy định** (khác với chuỗi ký của thanh toán); mỗi lời gọi có `vnp_RequestId` duy nhất, timeout `PRM-VNPAY_TIMEOUT`, thử lại tối đa 2 lần với lỗi mạng, bọc circuit breaker (BR-N Chịu lỗi dịch vụ ngoài). Giao dịch vẫn `PENDING` sau khi quá hạn và truy vấn xác nhận không có kết quả thì chuyển `EXPIRED`.

5.6 Hoàn tiền về nguồn

Mặc định hoàn vào ví (tức thì, bút toán ở mục 3.4.3). Hoàn về nguồn VNPay chỉ khi người gửi yêu cầu và giao dịch gốc là thanh toán trực tiếp qua VNPay (BR-R Nơi hoàn tiền):

1. Kiểm tra giao dịch gốc `SUCCESS`, số tiền yêu cầu $\leq$ số còn có thể hoàn; tạo `refund_requests` trạng thái chờ.
2. Tạm giữ tiền trong sổ cái: `Dr USER_AVAILABLE`, `Cr USER_PENDING_WD` (`ref_type = REFUND_TO_SOURCE`).
3. Gọi API hoàn tiền (toàn phần hoặc một phần theo tài liệu VNPay) với `vnp_RequestId` duy nhất; lưu mã phản hồi.
4. Thành công: `Dr USER_PENDING_WD`, `Cr VNPAY_CLEARING`, giao dịch gốc sang `REFUNDED` hoặc `PARTIAL_REFUNDED`. Chưa rõ kết quả: job truy vấn lại. Thất bại: đảo bước 2, tiền về ví và thông báo Admin.

Dòng 'Hoàn về nguồn VNPay' ở bảng 3.4.3 là bước 4 cộng bước 2 của quy trình này. VNPay hoàn về đúng phương thức thanh toán ban đầu; thời gian tiền về tài khoản người dùng phụ thuộc ngân hàng và hiển thị cho người dùng là 'đang xử lý'.

5.7 Rút tiền qua PayoutProvider

```
public interface PayoutProvider {
    String code(); // MOCK | MANUAL | tên nhà
    cung cấp thật
    PayoutResult submit(PayoutRequest request); // idempotent theo
    withdrawalId
    PayoutResult query(String payoutRef);
}
```

Triển khai	Hành vi	Dùng ở
MockPayoutProvider	Trả PROCESSING, tự chuyển PAID sau 30 giây (cấu hình được); số tiền thực nhận kết thúc bằng 13 giả lập FAILED để kiểm thử	local, dev, demo
ManualPayoutProvider	submit chỉ đánh dấu PROCESSING và đưa vào danh sách xuất; Admin chuyển khoản ngoài hệ thống rồi nhập mã tham chiếu (mark-paid) hoặc nhập tệp kết quả	Demo kịch bản thủ công
Nhà cung cấp chi hộ thật	Gọi API chi và nhận kết quả qua webhook hoặc truy vấn định kỳ	Thương mại hóa (sheet 12 phương án A, B)

Chọn nhà cung cấp bằng cấu hình `payout.provider`; đối nhà cung cấp **không** được dựng vào máy trạng thái yêu cầu rút hay số cái.

Quy tắc:

- Dữ liệu gửi đi: `withdrawalId`, số tiền thực nhận, mã ngân hàng, số tài khoản (giải mã chỉ trong bộ nhớ), tên chủ tài khoản, nội dung FPTPost rút tiền <id>. Số tài khoản không bao giờ vào log.
- `submit` idempotent: đã có `payout_ref` hoặc trạng thái đã là PROCESSING thì không gọi lại.
- Duyệt hai cấp và tách biệt nhiệm vụ do tầng ứng dụng kiểm soát trước khi gọi `submit` (BR-R Tách biệt nhiệm vụ khi duyệt rút).
- Chuyển trạng thái yêu cầu rút và bút toán `wd-paid` / `wd-reverse` tuân theo bảng ở sheet 08 và mục 3.4.3.
- Tệp xuất cho chi thủ công (CSV): `withdrawal_id`, `bank_code`, `account_no`, `holder_name`, `amount`, `memo`; tệp nhập kết quả: `withdrawal_id`, `status`, `bank_ref`, `paid_at`; dòng nhập không khớp trạng thái hiện tại bị bỏ qua và báo lỗi theo dòng.

5.8 Kịch bản kiểm thử VNPay (CT-16)

Thẻ và tài khoản thử nghiệm lấy từ trang sandbox của VNPay, không ghi cứng trong tài liệu hay mã nguồn.

Kịch bản	Kết quả mong đợi
Thanh toán thành công	Giao dịch SUCCESS; ví tăng đúng số tiền; một bút toán

Kịch bản	Kết quả mong đợi
Thanh toán thất bại (thẻ lỗi)	FAILED ; ví không đổi
Người dùng hủy ở trang VNPay	FAILED hoặc EXPIRED theo kết quả VNPay; ví không đổi
Bỏ trang, không thanh toán	Hết PRM-VNPAY_EXPIRE → EXPIRED ; ví không đổi
IPN gửi lặp 5 lần	Chỉ một bút toán; các lần sau phản hồi 02
IPN sai chữ ký	Phản hồi 97 ; không đổi trạng thái
IPN sai số tiền	MISMATCH ; phản hồi 04 ; cảnh báo Admin
IPN đến sau Return URL và ngược lại	Cùng một kết quả cuối
Mất IPN	Job truy vấn chốt trạng thái trong ≤ 4 phút
Thanh toán trực tiếp cho đơn đã bị hủy	Tiền được ghi có vào ví người gửi, thông báo
Hoàn tiền về nguồn thành công, thất bại, chưa rõ	Đúng ba nhánh ở mục 5.6; không mất tiền ở nhánh nào
50 yêu cầu nạp trùng Idempotency-Key	Chỉ một giao dịch được tạo

6. Thiết kế API

Danh sách endpoint, quyền truy cập và UC nằm ở sheet 10. Mục này quy định các quy ước chung mà mọi endpoint **PHẢI** theo.

6.1 Quy ước chung

Hạng mục	Quy ước
Địa chỉ gốc và phiên bản	/api/v1 ; thay đổi không tương thích tạo /api/v2 ; thêm trường tùy chọn không đổi phiên bản
Định dạng	JSON UTF-8; tên trường camelCase ; thời gian ISO-8601 UTC (2026-10-09T03:15:00Z); tiền là số nguyên VND ở trường amount ; định danh công khai là publicId (UUID)

Hạng mục	Quy ước
Phân trang	<code>page</code> (từ 0), <code>size</code> (mặc định 20, tối đa 100), <code>sort=trường,asc desc</code> ; phản hồi <code>{content, page, size, totalElements, totalPages}</code> . Chat và thông báo dùng con trỏ <code>before=<cursor></code>
Lọc	Tham số truy vấn tường minh (<code>status=OPEN&maxWeight=5</code>), không nhận biểu thức tự do
Xác thực	Header <code>Authorization: Bearer <accessToken></code>
Idempotency	Header <code>Idempotency-Key</code> bắt buộc cho POST có tác động tiền: nạp tiền, thanh toán ký quỹ, tạo yêu cầu rút, hoàn về nguồn, duyệt rút, giải ngân thủ công. Lưu trong <code>idempotency_keys</code> (hết hạn theo <code>PRM-IDEM-TTL</code>). Cùng khóa và cùng nội dung: trả lại phản hồi cũ; cùng khóa nhưng nội dung khác: <code>422 IDEMPOTENCY_KEY_REUSED</code>
Cập nhật đồng thời	<code>ETag</code> và <code>If-Match</code> (từ <code>version</code>) cho <code>PUT</code> đơn và chuyển đi; sai phiên bản trả <code>412</code>
Giới hạn tần suất	Vượt ngưỡng trả <code>429</code> kèm <code>Retry-After</code>
Theo vết	<code>X-Correlation-Id</code> (sinh nếu thiếu), có trong mọi phản hồi và mọi dòng log
CORS	Chỉ cho origin của frontend theo từng môi trường; không dùng <code>*</code>
Mã HTTP	<code>200/201/204</code> thành công; <code>400</code> dữ liệu sai; <code>401</code> chưa xác thực; <code>403</code> không đủ quyền; <code>404</code> không thấy hoặc không thuộc quyền (không lộ sự tồn tại); <code>409</code> xung đột trạng thái; <code>412</code> sai phiên bản; <code>422</code> vi phạm quy tắc nghiệp vụ; <code>429</code> ; <code>500</code> ; <code>503</code>

6.2 Định dạng lỗi

Mọi lỗi dùng chuẩn Problem Details (RFC 7807), thêm `code` ổn định và `correlationId`:

```
{
  "type": "https://fptpost.vn/errors/order-state-conflict",
  "title": "Đơn không ở trạng thái cho phép",
  "status": 409,
  "code": "ORDER_STATE_CONFLICT",
  "detail": "Đơn đang ở trạng thái ASSIGNED nên không thể thực hiện thao tác này.",
  "correlationId": "7f3c9a52-1d4e-4b8a-9c21-0a6e5d3f7b10",
  "errors": [{ "field": "weightKg", "code": "MAX_WEIGHT", "message": "Vượt giới hạn khối lượng" }]
}
```

`title` và `detail` tiếng Việt lấy từ tệp tài nguyên theo khóa là `code`; **cấm** đưa stack trace, câu SQL hoặc tên lớp vào phản hồi. Frontend ánh xạ theo `code`, không theo nội dung chữ.

Nhóm mã lỗi	Ví dụ (HTTP)
<code>AUTH_*</code>	<code>AUTH_INVALID_CREDENTIALS</code> (401), <code>AUTH_ACCOUNT_LOCKED</code> (423), <code>AUTH_OTP_INVALID</code> (422), <code>AUTH_UNVERIFIED</code> (403)
<code>KYC_*</code>	<code>KYC_REQUIRED</code> (403), <code>KYC_ID_DUPLICATED</code> (422)
<code>ORDER_*</code>	<code>ORDER_LIMIT_EXCEEDED</code> (422), <code>ORDER_STATE_CONFLICT</code> (409), <code>ILLEGAL_STATE_TRANSITION</code> (409), <code>COURIER_NOT_ELIGIBLE</code> (422)
<code>OFFER_*</code>	<code>OFFER_OUT_OF_BAND</code> (422), <code>OFFER_EXPIRED</code> (409)
<code>PAY_*</code>	<code>PAY_INSUFFICIENT_BALANCE</code> (422), <code>PAY_LIMIT_EXCEEDED</code> (422), <code>PAY_HOLD_EXPIRED</code> (409), <code>PAY_SIGNATURE_INVALID</code> (400)
<code>WD_*</code>	<code>WD_BELOW_MIN</code> (422), <code>WD_BANK_NAME_MISMATCH</code> (422), <code>WD_COOLDOWN_ACTIVE</code> (422), <code>WD_ALREADY_OPEN</code> (409)
<code>DISPUTE_*</code> , <code>SAFETY_*</code>	<code>DISPUTE_WINDOW_CLOSED</code> (422), <code>SAFETY_PROHIBITED_ITEM</code> (422)
<code>SYS_*</code>	<code>IDEMPOTENCY_KEY_REUSED</code> (422), <code>RATE_LIMITED</code> (429), <code>UPSTREAM_UNAVAILABLE</code> (503)

6.3 Ví dụ

```
POST /api/v1/wallet/topups
Authorization: Bearer <accessToken>
Idempotency-Key: 3b0f6b3a-7c1e-4a55-9d3e-2f9a0c8b1e42
Content-Type: application/json
```

```
{ "amount": 200000 }
```

```
{
  "txnRef": "TP261009103045A7K2QX",
  "paymentUrl": "<địa chỉ cổng VNPAY kèm tham số đã ký>",
  "expiresAt": "2026-10-09T03:30:45Z"
}
```

6.4 Hợp đồng OpenAPI

- `springdoc-openapi` sinh `openapi.json` ở mỗi build; CI kiểm tra spec hợp lệ và so sánh thay đổi phá vỡ (`openapi-diff`) với nhánh chính.
- Mỗi endpoint có `@Operation(summary)` và `@ApiResponse` cho các mã lỗi chính, gom nhóm theo module; yêu cầu quyền ghi trong mô tả.
- Hợp đồng được chốt trước mỗi sprint; khi backend chưa xong, frontend dùng máy chủ giả lập dựng từ OpenAPI (ví dụ Prism) để không bị chặn.

7. Thiết kế bảo mật

7.1 Xác thực

Hạng mục	Thiết kế
Mật khẩu	BCrypt cost 12; kiểm tra độ mạnh theo <code>PRM-PWD_MIN</code> ; không giới hạn ký tự đặc biệt; không bao giờ ghi log
Access Token	JWT ký HS256 bằng khóa ≥ 256 bit lấy từ biến môi trường; claim: <code>sub</code> (<code>publicId</code>), <code>roles</code> , <code>sid</code> (phiên), <code>jti</code> , <code>iat</code> , <code>exp</code> (<code>PRM-JWT_ACCESS</code>). Frontend giữ trong bộ nhớ (React state), không lưu <code>localStorage</code>

Hạng mục	Thiết kế
Refresh Token	Chuỗi ngẫu nhiên 256 bit, lưu SHA-256 trong sessions; gửi qua cookie HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth, hạn PRM-JWT_REFRESH
Xoay vòng	Mỗi lần refresh cấp token mới và đánh dấu token cũ replaced_by. Dùng lại token cũ thì thu hồi mọi phiên của tài khoản (BR-R Vòng đời token)
Khóa đăng nhập	Bộ đếm theo tài khoản và theo IP; PRM-LOGIN_MAX_FAIL lần sai khóa PRM-LOGIN_LOCK; đúng trước đó thì đặt lại bộ đếm
Thông điệp lỗi	Sai thông tin luôn trả cùng một lỗi, không nói trường nào sai; so sánh mật khẩu với băm giả khi tài khoản không tồn tại để thời gian phản hồi không lộ tài khoản
Giới hạn thiết bị	Đăng nhập thứ PRM-MAX_DEVICES +1 thu hồi phiên cũ nhất
Giới hạn tần suất	Nginx limit_req ở rìa cộng Bucket4j trong ứng dụng (PRM-RATE_API , PRM-RATE_AUTH); chấp nhận sai số nhỏ khi chạy hai instance vì bộ đếm theo từng instance

7.2 Phân quyền

- **RBAC:** các quyền ROLE_SENDER, ROLE_COURIER, ROLE_MODERATOR, ROLE_ADMIN; kiểm tra bằng @PreAuthorize ở lớp service. Ma trận ở sheet 02.
- **Kiểm tra sở hữu (chống BOLA/IDOR):** không bao giờ tin mã định danh từ client. Tải đối tượng theo cặp (publicId, userId) hoặc kiểm tra tư cách tham gia ngay sau khi tải (@orderSecurity.isParticipant(...)); không thuộc quyền thì trả 404.
- **Người giao đủ điều kiện:** chú thích @RequiresVerifiedCourier kiểm tra vai trò, KYC đã duyệt và tài khoản không bị hạn chế trước khi vào nghiệp vụ nhận đơn.
- **Duyệt hai cấp:** tầng ứng dụng so sánh hai người duyệt khác nhau và không phải người tạo yêu cầu (BR-R Tách biệt nhiệm vụ).
- **Endpoint công khai** được liệt kê tường minh trong một lớp cấu hình duy nhất: đăng ký/ đăng nhập/làm mới/OTP/quên mật khẩu, ước tính giá, Return URL và IPN của VNPay, theo dõi công khai theo token, actuator/health. Mọi đường dẫn khác mặc định cần xác thực. IPN không dùng JWT mà dựa vào chữ ký VNPay; có thể thêm danh sách IP cho phép ở Nginx nếu VNPay công bố.

- **Tài khoản nhân sự** (Kiểm duyệt, Admin) tạo bởi Admin, bắt buộc đổi mật khẩu lần đầu; mọi thao tác nhạy cảm ghi `audit_logs`.

7.3 Bảo vệ dữ liệu

Dữ liệu	Biện pháp
Số CCCD, số tài khoản ngân hàng	Mã hóa trường AES-256-GCM; lưu <code>key_id</code> , IV và bản mã. Khóa dữ liệu bọc bởi khóa chủ lấy từ biến môi trường hoặc kho bí mật. Tra cứu trùng CCCD bằng HMAC-SHA256(<code>K_hash</code> , số đã chuẩn hóa)
Ảnh KYC	Lưu mã hóa; chỉ truy cập qua liên kết ký có hạn 5 phút; mỗi lượt xem ghi <code>audit_logs</code> (BR-R Quyền xem dữ liệu nhạy cảm)
Hiển thị	Số điện thoại che dạng <code>09*****12</code> ; số tài khoản chỉ hiện 4 số cuối
Log	Có bộ lọc che dữ liệu nhạy cảm; không ghi nội dung yêu cầu/phản hồi của <code>/auth</code> , <code>/wallet</code> , <code>/me/kyc</code>
Tệp tải lên	Kiểm tra loại theo nội dung (không tin phần mở rộng), giới hạn <code>PRM-MAX_PHOTO_MB</code> , đổi tên ngẫu nhiên, ghi lại ảnh để loại siêu dữ liệu EXIF với ảnh hàng và ảnh đại diện (tọa độ bằng chứng lưu ở cột riêng)
Đường truyền	TLS 1.2 trở lên, HSTS; WebSocket dùng WSS

7.4 Đối chiếu OWASP Top 10 (2021)

Rủi ro	Biện pháp trong thiết kế
A01 Kiểm soát truy cập	Từ chối mặc định, kiểm tra sở hữu, 404 thay 403 cho đối tượng ngoài quyền, kiểm thử BOLA tự động
A02 Lỗi mật mã	BCrypt, AES-GCM, HMAC, TLS; khóa ngoài mã nguồn; xoay khóa theo <code>kid</code>
A03 Injection	JPA/JSQL có tham số; Mongo dùng đối tượng truy vấn có kiểu; kiểm tra đầu vào bằng Bean Validation; mã hóa đầu ra ở React
A04 Thiết kế không an toàn	Số cái bất biến, idempotency, duyệt hai cấp, giới hạn hạn mức, mô hình đe dọa ở sprint 1

Rủi ro	Biện pháp trong thiết kế
A05 Cấu hình sai	Cấu hình theo profile, tắt endpoint Actuator ngoài health trên mạng ngoài, header bảo mật, không bật Swagger ở prod
A06 Thành phần lỗi thời	Dependabot hàng tuần, OWASP Dependency-Check, npm audit, quét ảnh Docker bằng Trivy
A07 Lỗi xác thực	Mục 7.1; xoay vòng refresh token; khóa tạm; không lộ tài khoản
A08 Toàn vẹn phần mềm và dữ liệu	CI bắt buộc, kiểm tra chữ ký IPN, khóa phiên bản phụ thuộc, không chạy mã từ nguồn không tin cậy
A09 Ghi log và giám sát	Correlation ID, audit_logs, cảnh báo job lỗi và vi phạm bất biến số cái
A10 SSRF	RoutingClient chỉ gọi tên miền trong danh sách cho phép; không nhận URL từ người dùng để tải về

7.5 Header và chính sách trình duyệt

Content-Security-Policy: default-src 'self'; img-src 'self' data: https://*.tile.openstreetmap.org; connect-src 'self' wss:; X-Content-Type-Options: nosniff; X-Frame-Options: DENY; Referrer-Policy: strict-origin-when-cross-origin; Permissions-Policy: geolocation=(self), camera=(self). API xác thực bằng Bearer nên không chịu CSRF; endpoint refresh dựa vào cookie SameSite=Strict và kiểm tra header Origin.

7.6 Bí mật và phụ thuộc

Bí mật đi qua biến môi trường hoặc Docker secrets, và GitHub Actions secrets trong CI; công cụ quét bí mật (gitleaks) chạy trong CI trên mọi pull request và chặn merge nếu phát hiện. Khóa production của VNPay và khóa mã hóa **không** được tồn tại ở máy phát triển.

8. Thời gian thực và thông báo

8.1 WebSocket (STOMP)

- Điểm kết nối /ws qua WSS. Token truy cập gửi trong header Authorization của khung STOMP CONNECT (**không** đặt token trên URL). Interceptor xác thực và gán Principal; token hết hạn thì server ngắt, client làm mới token rồi kết nối lại.

- Mỗi khung `SUBSCRIBE` đều được kiểm tra quyền: chỉ bên tham gia đơn (hoặc Kiểm duyệt/Admin) mới đăng ký được kênh của đơn đó; sai quyền trả khung `ERROR`.

Kênh	Hướng	Nội dung
<code>/app/chat.{orderId}.send</code>	Client → server	Gửi tin nhắn; server kiểm tra, lưu MongoDB rồi phát lại
<code>/topic/chat.{orderId}</code>	Server → client	Tin nhắn mới, trạng thái đã đọc
<code>/topic/track.{orderId}</code>	Server → client	Vị trí mới nhất, mốc hành trình, ETA
<code>/user/queue/notifications</code>	Server → client	Thông báo riêng của người dùng
<code>/user/queue/orders</code>	Server → client	Thay đổi trạng thái của đơn liên quan đến tôi

Phong bì tin: `{id, type, orderId, payload, at, v}`; client bỏ qua tin có `id` đã xử lý. Nhịp tim 10 giây hai chiều. Mật kết nối: kết nối lại với backoff lũy tiến 1 → 30 giây có nhiều ngẫu nhiên, **sau đó tải bù bằng REST** (`GET /orders/{id}/tracking`, tin nhắn sau `lastId`) để không mất sự kiện. Người nhận mở liên kết công khai không có WebSocket: trang thăm dò mỗi 5 giây.

Với broker nội bộ của Spring, tin phát ở instance này không tới người đăng ký ở instance khác. Trong phạm vi đồ án: các instance REST chạy bình thường, **một instance mang profile realtime** giữ broker và chạy nhóm xử lý outbox đẩy WebSocket; mở rộng thật sự sẽ chuyển sang broker relay (OQ-03).

8.2 Luồng vị trí

- Frontend gọi `navigator.geolocation.watchPosition` (độ chính xác cao), gom điểm và gửi `POST /orders/{id}/locations` theo lô mỗi `PRM-GPS_INTERVAL`; dùng Screen Wake Lock khi đang giao; khi mất mạng xếp hàng cục bộ tối đa 200 điểm và gửi bù theo `seq`.
- Backend kiểm tra đơn đang `PICKED_UP` / `IN_TRANSIT` và người gọi là người giao; **loại** điểm có độ chính xác > 100 m hoặc vận tốc phi lý (> 120 km/h); ghi `location_pings`; phát `/topic/track.{orderId}` với điểm mới nhất (gộp tối đa một bản tin mỗi giây).
- ETA = quãng đường còn lại tới điểm giao (định tuyến, cache 30 giây) chia vận tốc trung bình 5 phút gần nhất, tối thiểu 10 km/h.
- Ngừng nhận điểm ngay khi đơn rời `IN_TRANSIT` (BR-R Phạm vi chia sẻ vị trí). Hạn chế chạy nền của trình duyệt được nêu ở BR-C Phạm vi trình duyệt và rủi ro R-07.

8.3 Đường ống thông báo

Sự kiện miền → `outbox_events` → `NotificationDispatcher` → chọn mẫu theo `type` và ngôn ngữ → áp dụng cài đặt người dùng (nhóm bắt buộc không tắt được) → gửi qua kênh → thử lại theo `PRM-NOTIF_RETRY` với khoảng chờ 1, 5, 30 phút → thất bại thì đánh dấu `failed` và cảnh báo.

Sự kiện	Trong ứng dụng	Email	SMS
OTP (đăng ký, đặt lại mật khẩu)	-	Có	Có (nếu đăng ký bằng SĐT)
Đơn đổi trạng thái	Có	Với <code>DELIVERED</code> , <code>COMPLETED</code> , <code>CANCELLED</code> , <code>EXPIRED</code>	OTP giao hàng cho người nhận
Nhận chào giá, chào giá được chấp nhận	Có	-	-
Thanh toán, hoàn tiền, giải ngân	Có	Có	-
Yêu cầu rút đổi trạng thái	Có	Có	-
Kết quả KYC	Có	Có	-
Khiếu nại mở, phán quyết, kháng nghị	Có	Có	-
SOS	Có (ưu tiên cao cho Kiểm duyệt)	-	Người liên hệ khẩn cấp (nếu người dùng đã đăng ký)
Lên/xuống hạng	Có	-	-

Mỗi thông báo có `dedupeKey` = `eventId` + `kênh` để chạy lại outbox không tạo thông báo trùng (mục 3.5).

9. Thiết kế frontend

Danh sách màn hình và đường dẫn ở sheet 11; mục này quy định cấu trúc và quy ước để bốn người làm frontend và full-stack viết nhất quán.

9.1 Cấu trúc dự án

```
src/
├─ app/          router, providers (Query, Auth, Ws, I18n), layout theo chế độ
├─ features/     auth, profile, kyc, orders, market, trips, tracking, chat,
│   │           wallet, disputes, reputation, notifications, admin
│   └─ <feature>/ api/ hooks/ components/ pages/ schema/
├─ shared/       ui/ (bọc React-Bootstrap), hooks/, lib/ (http, ws, geo,
format), i18n/
└─ main.jsx
```

Quy tắc: `features/*` chỉ gọi nhau qua `shared/` hoặc qua hook công khai của feature; không import trực tiếp tệp nội bộ của feature khác. Mọi lời gọi API nằm trong `features/<f>/api`, không gọi `axios` trong component.

9.2 Điều hướng và bảo vệ route

React Router 6 với các lớp bọc `RequireAuth`, `RequireRole(roles)`, `RequireVerified` (đã xác thực OTP) và `RequireKyc`. Route tải lười theo vai trò để bundle ban đầu ≤ 300 KB gzip. Người dùng có cả hai vai trò dùng bộ chuyển chế độ (Người gửi/Người giao) ở thanh điều hướng; chế độ lưu trong Context và `sessionStorage`, không ảnh hưởng quyền thật vốn do backend quyết định.

9.3 Dữ liệu và bộ nhớ đệm (TanStack Query)

Quy ước	Nội dung
Khóa truy vấn	<code>['orders', filters]</code> , <code>['order', publicId]</code> , <code>['market', filters]</code> , <code>['wallet']</code> , <code>['notifications']</code> ; tạo bằng hàm <code>keys.*</code> tập trung
<code>staleTime</code>	Sàn đơn 15 giây; chi tiết đơn 5 giây; ví 10 giây; hồ sơ và danh mục 5 phút
Làm mới theo sự kiện	Tin WebSocket <code>order.updated</code> → <code>invalidateQueries(['order', id])</code> và <code>['orders']</code> ; <code>wallet.changed</code> → <code>['wallet']</code>
Cập nhật lạc quan	Chỉ dùng cho đánh dấu thông báo đã đọc; cấm dùng cho thao tác tiền và đổi trạng thái đơn
Mutation có tác động tiền	Hook <code>useIdempotentMutation</code> sinh <code>Idempotency-Key</code> (UUID) một lần mỗi lần mở biểu mẫu và giữ nguyên khi thử lại; nút gửi bị khóa trong lúc chờ

Quy ước	Nội dung
Giới hạn nghiệp vụ trên biểu mẫu	Lấy từ khối <code>limits</code> trong phản hồi <code>POST /pricing/estimate</code> (khối này phải có trong hợp đồng OpenAPI) thay vì viết cứng

9.4 HTTP client

`axios` một instance với `baseUrl=/api/v1`, `timeout 15` giây; interceptor gắn `Bearer` và `X-Correlation-Id`. Gặp `401`: gọi làm mới token **một lần** và xếp hàng các yêu cầu đang chờ; làm mới lỗi thì đăng xuất và về trang đăng nhập. Lỗi `Problem Details` được chuyển thành `ApiError{status, code, fieldErrors, correlationId}`; toast lỗi lấy câu chữ từ từ điển `i18n` theo `code` và có nút sao chép `correlationId` cho hỗ trợ. Hủy yêu cầu khi component bị gỡ bằng `AbortController`.

9.5 Biểu mẫu và kiểm tra

React Hook Form kết hợp `Zod`; kiểm tra phía client chỉ để trải nghiệm tốt, **backend vẫn là nơi kiểm tra quyết định**. Số tiền nhập có định dạng nghìn nhưng gửi lên là số nguyên. Biểu mẫu tạo đơn là wizard nhiều bước lưu nháp cục bộ để không mất dữ liệu khi tải lại trang; ảnh được nén phía client (cạnh dài ≤ 1600 px, JPEG chất lượng 0,8) trước khi tải lên.

9.6 Bản đồ

React-Leaflet với `OpenStreetMap`, luôn hiển thị ghi công (attribution). Gợi ý địa chỉ qua `/geo/search` với `debounce 400 ms` và tối thiểu 3 ký tự; sàn đơn dạng bản đồ dùng gom cụm marker; trang theo dõi di chuyển marker mượt giữa hai điểm vị trí và hiển thị ETA bằng chữ. Không gọi dịch vụ định tuyến trực tiếp từ trình duyệt; mọi truy vấn đi qua backend để dùng chung bộ nhớ đệm và danh sách tên miền cho phép.

9.7 Giao diện và quốc tế hóa

React-Bootstrap, thiết kế ưu tiên di động từ 360 px; mỗi danh sách có đủ ba trạng thái (đang tải dạng skeleton, rỗng, lỗi có nút thử lại). Tiền hiển thị bằng `Intl.NumberFormat('vi-VN')` kèm ký hiệu đ, ngày `dd/MM/yyyy`, giờ 24 giờ. Mọi chuỗi giao diện nằm trong tệp JSON của `react-i18next` (mặc định `vi`); **cấm** chuỗi tiếng Việt viết cứng trong JSX. Tuân thủ mức cơ bản WCAG 2.1 AA: nhấn cho mọi ô nhập, tương phản đủ, điều hướng được bằng bàn phím.

9.8 Client thời gian thực

`@stomp/stompjs` trong một `WsProvider` duy nhất; trang đăng ký kênh khi mở và hủy khi rời; kết nối lại theo backoff của mục 8.1, **sau mỗi lần kết nối lại** làm mới các truy vấn liên quan để bù dữ liệu bị lỗi. Trạng thái kết nối hiển thị bằng dải thông báo nhỏ khi mất mạng.

10. DevOps và vận hành

10.1 Docker Compose

Dịch vụ	Mô tả	Ghi chú
<code>nginx</code>	Reverse proxy, TLS, phục vụ tệp tĩnh của frontend, <code>limit_req</code>	Cổng 80/443; định tuyến <code>/api</code> và <code>/ws</code> về backend
<code>backend</code>	Spring Boot 3, JRE 21	Cổng 8080 nội bộ; healthcheck <code>/actuator/health</code> ; chạy bằng người dùng không phải root; <code>-XX:MaxRAMPercentage=75</code>
<code>sqlserver</code>	SQL Server (bản Linux)	Cổng 1433; volume dữ liệu; cần chấp nhận EULA qua biến môi trường và đủ RAM theo khuyến nghị của nhà cung cấp
<code>mongo</code>	MongoDB	Cổng 27017; volume dữ liệu
<code>minio</code>	Lưu trữ tệp tương thích S3	Cổng 9000/9001; bucket riêng cho ảnh đơn, bằng chứng, KYC
<code>mailpit</code>	Hộp thư giả lập (dev)	Cổng 1025/8025
<code>tunnel</code>	Tunnel công khai cho IPN (dev)	cloudflared hoặc ngrok, trỏ tới <code>nginx</code>

Dockerfile backend nhiều giai đoạn (build bằng Maven, chạy trên JRE 21). Frontend build bằng Node 21 rồi chép tệp tĩnh vào image `nginx`. Một số máy kiến trúc ARM không chạy trực tiếp được image SQL Server; thành viên gặp trường hợp này dùng SQL Server chung trên máy chủ dev của nhóm (rủi ro ghi ở R-14).

10.2 CI/CD (GitHub Actions)

Giai đoạn	Việc	Chặn khi
1. Định dạng	Checkstyle/Spotless; ESLint/Prettier	Có lỗi

Giai đoạn	Việc	Chặn khi
2. Build và kiểm thử đơn vị	<code>mvn verify</code> kèm JaCoCo; <code>vitest</code>	Test đỏ hoặc phủ < 70% ở module tiền, đơn
3. Kiểm thử tích hợp	Testcontainers (SQL Server, MongoDB)	Test đỏ
4. Kiến trúc và hợp đồng	ArchUnit; kiểm tra OpenAPI hợp lệ và thay đổi phá vỡ	Vi phạm
5. Bảo mật	gitleaks, OWASP Dependency-Check, <code>npm audit</code> , Trivy quét image	Phát hiện mức Cao/Nghiêm trọng hoặc bí mật
6. Đóng gói	Build image, gắn thẻ bằng mã commit, đẩy lên registry	Lỗi build
7. Triển khai	<code>main</code> tự động lên <code>dev</code> ; thẻ <code>rc-*</code> lên <code>staging</code> sau khi một người duyệt thủ công	-
8. Kiểm thử khối	Playwright chạy luồng chính trên môi trường vừa triển khai	Test đỏ thì báo và dừng đưa lên <code>staging</code>

Pull request chỉ được merge khi toàn bộ giai đoạn 1-5 xanh và có ít nhất một người duyệt (BR-N Quy ước và review).

10.3 Quan sát

- **Log:** JSON có cấu trúc gồm `ts`, `level`, `correlationId`, `module`, `event`, `userRef` (bắt, không phải định danh thật); không có dữ liệu cá nhân; mức `INFO` cho nghiệp vụ, `WARN` cho hành vi bất thường, `ERROR` kèm `correlationId`.
- **Chỉ số:** Actuator và Micrometer; các chỉ số cốt lõi: thời gian xử lý HTTP (p95), `payment_ipn_total{result}`, `ledger_post_total`, thời lượng và kết quả từng job, số kết nối WebSocket, độ trễ outbox. Prometheus và Grafana là tùy chọn.
- **Cảnh báo (gửi cho Admin):** có IPN `MISMATCH`; vi phạm bất biến số cái; job thất bại; độ trễ outbox > 60 giây; p95 API > 1 giây trong 5 phút.
- **Sức khỏe:** `liveness` và `readiness` kiểm tra kết nối SQL Server và MongoDB.

10.4 Sao lưu, khôi phục, phát hành

- Sao lưu hằng ngày: SQL Server (full), MongoDB (`mongodump`), MinIO (đồng bộ); giữ 7 bản; mục tiêu RPO ≤ 24 giờ và RTO ≤ 4 giờ; diễn tập khôi phục mỗi tháng và lưu biên bản (CT-27).

- Phiên bản theo `MAJOR.MINOR.PATCH`, gắn thẻ Git cho mỗi bản phát hành kèm changelog.
- Migration chạy khi backend khởi động (Flyway). Thay đổi cấu trúc theo hai bước **mở rộng rồi thu hẹp**: thêm cột/bảng ở bản này, chỉ xóa phần cũ ở bản sau để quay lui ứng dụng không làm hỏng dữ liệu.

11. Chiến lược kiểm thử

11.1 Các tầng kiểm thử

Tầng	Công cụ	Phạm vi	Chỉ tiêu
Đơn vị backend	JUnit 5, Mockito	Quy tắc miền, máy trạng thái, tính giá, số cái	Phủ $\geq 70\%$ module tiên và đơn
Tích hợp backend	Spring Boot Test, Testcontainers (SQL Server, MongoDB), REST Assured	Mọi API tiên, mọi chuyển trạng thái, phân quyền	Chạy trong CI, không dùng DB giả
Hợp đồng API	Kiểm tra phản hồi theo OpenAPI trong test	Khớp spec với thực tế	0 lệch
Đơn vị frontend	Vitest, Testing Library	Hook, kiểm tra biểu mẫu, ánh xạ lỗi	Các hook và biểu mẫu chính
Đầu-cuối (E2E)	Playwright (hoặc Cypress)	8 luồng ở mục 11.4	Xanh trên <code>dev</code> sau mỗi lần triển khai
Hiệu năng	k6	Duyệt sàn, tạo đơn, nhận vị trí, bảo IPN	Đạt các chỉ tiêu BR-N Hiệu năng
Bảo mật	OWASP ZAP (quét cơ sở), danh sách kiểm OWASP, kiểm thử BOLA tự động	Toàn bộ API	0 lỗ hổng Cao/Nghiêm trọng
Chấp nhận (UAT)	Kịch bản theo UC mức Must	Người dùng thật của nhóm	Biên bản đạt

11.2 Kiểm thử bắt buộc cho tiên và trạng thái

1. **Bảng chuyển trạng thái đầy đủ**: duyệt mọi cặp (trạng thái $\times$ sự kiện); cặp ngoài sheet 08 phải trả `ILLEGAL_STATE_TRANSITION`.

2. **Bất biến số cái:** kiểm thử dựa trên thuộc tính (ví dụ jqwik) chạy chuỗi nghiệp vụ ngẫu nhiên (nạp, ký quỹ, giải ngân, hoàn, rút, bồi thường) và sau mỗi bước khẳng định bốn bất biến ở mục 3.4.3.
3. **Đồng thời:** 50 luồng cùng nhận một đơn chỉ một thắng; 50 yêu cầu nạp trùng `Idempotency-Key` chỉ tạo một giao dịch; IPN gửi lặp chỉ ghi một bút toán.
4. **Làm tròn:** 20 ca ở sheet 07 (tệp `pricing-cases.csv`).
5. **Thời gian:** inject `Clock` ở mọi nơi dùng giờ hiện tại (**cấm** gọi `Instant.now()` trực tiếp trong nghiệp vụ) để kiểm thử hết hạn, giữ chỗ, quá hạn lấy hàng, tự giải ngân mà không phải chờ.
6. **Phân quyền:** với mỗi endpoint có định danh trong đường dẫn, kiểm thử người dùng khác nhận `404`.

11.3 Dữ liệu kiểm thử

Bộ seed theo kịch bản: 3 người gửi, 3 người giao ở các hạng khác nhau, 1 Kiểm duyệt, 2 Admin (để thử duyệt hai cấp), đơn ở mọi trạng thái, ví có số dư, tham số mặc định. Dùng builder/factory trong test, không dùng dữ liệu thật; ảnh mẫu là ảnh tự tạo.

11.4 Tám luồng E2E bắt buộc

#	Luồng	UC chính
1	Đăng ký → OTP → đăng nhập → đăng xuất	UC-01..04
2	Nộp KYC → Kiểm duyệt duyệt → đăng ký làm Người giao	UC-08..10
3	Tạo đơn → lên sàn → nhận đơn → ký quỹ → lấy hàng → giao bằng OTP → giải ngân → đánh giá	UC-12, 16, 17, 35, 25, 28, 36, 47
4	Chào giá → phản giá → chấp nhận → ký quỹ	UC-18, 19, 35
5	Nạp tiền qua VNPAY (CI dùng <code>MockPaymentGateway</code> , sandbox thật chạy thử công mỗi sprint)	UC-34, 42
6	Yêu cầu rút → duyệt hai cấp → chi (Mock)	UC-40, 41
7	Khiếu nại → phán quyết → bút toán	UC-50, 51
8	Hủy đơn ở từng thời điểm và kiểm tra hoàn tiền	UC-15, 37

11.5 Tiêu chí sẵn sàng và hoàn thành

Definition of Ready (UC được đưa vào sprint khi): có BR liên quan và tiêu chí nghiệm thu; endpoint đã nằm trong hợp đồng OpenAPI; có wireframe cho màn hình; phụ thuộc đã xong hoặc đã có giả lập.

Definition of Done (UC được coi là xong khi): mã đã được ít nhất một người review và merge; có kiểm thử mới đặt trong CI; lint và quét bảo mật xanh; OpenAPI và migration cập nhật; log, chỉ số cần thiết đã có; demo được trên `dev`; tiêu chí nghiệm thu của mọi BR liên quan đạt; nếu thay đổi hành vi so với đặc tả thì Excel hoặc SPEC đã được cập nhật.

12. Quy ước phát triển

12.1 Git và review

Hạng mục	Quy ước
Nhánh	<code>main</code> luôn triển khai được và được bảo vệ; <code>feature/UC-12-tao-don</code> , <code>fix/...</code> , <code>hotfix/...</code> ; nhánh sống tối đa 3 ngày, rebase trên <code>main</code> trước khi mở PR, merge kiểu squash
Commit	Conventional Commits: <code>feat(order): UC-12 tạo đơn</code> , <code>fix(ledger): ...</code> , <code>test</code> , <code>docs</code> , <code>refactor</code> , <code>chore</code>
Pull request	Tối đa khoảng 400 dòng thay đổi (trừ migration và seed); mô tả nêu UC, BR, cách kiểm thử, ảnh chụp với thay đổi giao diện
Review	Ít nhất một người không phải tác giả, phản hồi trong 24 giờ làm việc; thay đổi ở module <code>ledger</code> , <code>payment</code> PHẢI có TV1 hoặc người dự phòng review (CODEOWNERS)
Người dự phòng theo module	TV1 ↔ TV3 (tiền, đơn), TV2 ↔ TV6 (xác thực, thời gian thực), TV4 ↔ TV5 (frontend)

12.2 Backend

- DTO dùng `record`; không trả thực thể JPA ra API; ánh xạ bằng MapStruct.
- Tiêm phụ thuộc qua constructor; không `@Autowired` lên trường; không đặt logic nghiệp vụ trong controller.

- `@Transactional` ở tầng service (`readOnly = true` cho truy vấn); không để giao dịch bao quanh lời gọi dịch vụ ngoài (VNPay, SMS).
- Tránh N+1: dùng `@EntityGraph` hoặc `fetch join` và có kiểm thử đếm số truy vấn cho các danh sách chính.
- Tiền là `long`, thời gian là `Instant` kèm `Clock` được inject; tuyệt đối không dùng `float/double` cho tiền.
- Lỗi: một `@RestControllerAdvice` duy nhất; ngoại lệ miền `BusinessException(code, HttpStatus)`; không bắt `Exception` rộng, không nuốt lỗi.
- Log bằng SLF4J với tham số `{}`; không nổi chuỗi; không log dữ liệu cá nhân.
- Tên: `XxxController`, `XxxService`, `XxxRepository`, `XxxMapper`, `XxxRequest`, `XxxResponse`; tên test `method_condition_expected`.
- Tham số nghiệp vụ lấy qua `ParameterService` (nạp từ bảng `system_parameters`, bộ nhớ đệm 60 giây, làm mới khi Admin đổi); **cấm** viết cứng số nghiệp vụ.

12.3 Frontend

- JavaScript ES2022 với JSDoc cho kiểu dữ liệu chính (chuyển TypeScript là tùy chọn nếu cả nhóm đồng ý, OQ-05); ESLint (khuyến nghị cho React và hook) và Prettier bắt buộc trong CI.
- Component hàm và hook; tệp component đặt tên PascalCase; hook bắt đầu bằng `use`; mỗi component một trách nhiệm, tách logic vào hook.
- Không dùng `localStorage` cho token hoặc dữ liệu nhạy cảm; không `dangerouslySetInnerHTML` với dữ liệu người dùng.

12.4 Quy ước đặt tên chung

Đường dẫn API `kebab-case`; JSON `camelCase`; cột và bảng CSDL `snake_case`; enum `UPPER_SNAKE_CASE`; biến môi trường `UPPER_SNAKE_CASE` với tiền tố `FPTPOST_`; khóa i18n dạng `feature.screen.element`.

13. Quản lý dự án

13.1 Nhịp làm việc

Sự kiện	Tần suất	Nội dung
Họp nhanh	Hằng ngày 15 phút (ít nhất 3 lần/tuần khi lịch học dày)	Hôm qua xong gì, hôm nay làm gì, đang bị chặn bởi gì

Sự kiện	Tần suất	Nội dung
Lập kế hoạch sprint	Đầu sprint, 60–90 phút	Chọn UC theo sheet 04 và 13; kiểm tra Definition of Ready; chốt người nhận việc
Đồng bộ API	Thứ Hai hằng tuần, 30 phút	Thay đổi OpenAPI, chặn nhau giữa frontend và backend
Tinh chỉnh backlog	Giữa sprint, 30 phút	Làm rõ UC của sprint sau
Review và demo	Cuối sprint, 60 phút	Demo trên <code>dev / staging</code> theo tiêu chí ở sheet 13
Retrospective	Cuối sprint, 45 phút	Điều gì tốt, điều gì sửa, hành động cụ thể có người phụ trách

13.2 Phân công trách nhiệm

Hoạt động	Chịu trách nhiệm cuối	Thực hiện	Tham vấn
Phạm vi và thay đổi BR	TV1	TV1, TV3	Cả nhóm
Kiến trúc và ADR	TV3	TV3, TV6	TV1, TV2
Sổ cái và VNPay	TV1	TV1	TV3, TV6 (review)
Bảo mật	TV2	TV2	TV6
UI kit và chuẩn frontend	TV4	TV4, TV5	TV2
CI/CD và môi trường	TV6	TV6	TV3
Kiểm thử và chất lượng	TV6	Mọi thành viên (kiểm thử đơn vị của phần mình), TV6 (E2E, tải)	TV2
Tài liệu (Excel, SPEC)	TV1	Mọi thành viên cập nhật phần của mình	TV3
Demo và kịch bản	TV1	TV4, TV5	Cả nhóm

13.3 Kiểm soát thay đổi

Yêu cầu thay đổi mở bằng issue nhãn `change-request`, nêu tác động đến BR, UC, giờ công, rủi ro; TV1 quyết định trong 2 ngày làm việc. Sau sprint 3 chỉ nhận thay đổi nếu đổi lại bằng việc cắt lượng giờ tương đương. Mọi thay đổi được thực hiện đồng thời ở Excel (BR chuyển trạng thái `Thay đổi`) và SPEC, kèm tăng số phiên bản tài liệu.

13.4 Rủi ro và chỉ số theo dõi

Rủi ro được rà soát đầu mỗi sprint theo sheet 15; rủi ro có điểm ≥ 15 phải có người phụ trách và phương án dự phòng. Chỉ số theo dõi mỗi sprint: giờ hoàn thành so với giờ kế hoạch (từ sheet 14), số lỗi mở theo mức nghiêm trọng, độ phủ kiểm thử, thời gian trung bình PR chờ review, số lần pipeline đỏ. Nếu giờ hoàn thành thấp hơn 80% kế hoạch hai sprint liên tiếp thì hợp cắt phạm vi theo MoSCoW ngay trong tuần đó.

Phụ lục A. Câu hỏi còn mở

Các điểm dưới đây là **quyết định thiết kế hoặc giả định cần nhóm xác nhận**; cột đề xuất là phương án SPEC đang dùng.

Mã	Câu hỏi	Đề xuất hiện tại	Hạn chót
OQ-01	Phí hủy đơn có bị thu hoa hồng không?	Chuyển toàn bộ phí hủy cho người giao, không thu hoa hồng (bút toán ở mục 3.4.3)	Trước sprint 3
OQ-02	Miền email nào được cấp huy hiệu Cộng đồng FPT?	Danh sách cấu hình, ví dụ <code>fpt.edu.vn</code> , <code>fpt.com</code> ; nhóm xác nhận danh sách	Trước sprint 2
OQ-03	WebSocket khi chạy nhiều instance	Một instance mang profile <code>realtime</code> giữ broker; mở rộng thật dùng broker relay	Trước sprint 3
OQ-04	Nơi lưu ảnh khi demo	MinIO trong Docker Compose; hệ thống tệp cục bộ nếu không đủ tài nguyên	Trước sprint 2
OQ-05	JavaScript hay TypeScript cho frontend	JavaScript + JSDoc theo đề cương; chuyển TypeScript nếu cả nhóm quen	Trước sprint 1
OQ-06	OTP bằng SMS thật hay giả lập khi demo	Giả lập (hiển thị ở hộp thư Mailpit/nhật ký); SMS thật cần nhà cung cấp và chi phí	Trước sprint 4

Mã	Câu hỏi	Đề xuất hiện tại	Hạn chót
OQ-07	Phạm vi địa lý ban đầu	Cộng đồng FPT và vùng quanh campus; ảnh hưởng danh mục khu vực và điểm hẹn	Trước sprint 2
OQ-08	Bảo hiểm có bắt buộc với đơn vượt PRM-HIGH_VALUE không	Tùy chọn, nhưng đơn chỉ giao cho người giao hạng Bạc trở lên	Trước sprint 3
OQ-09	Ai soạn Điều khoản dịch vụ và Chính sách dữ liệu	Nhóm soạn bản nháp, nhờ người có chuyên môn rà soát trước khi demo	Trước sprint 5
OQ-10	Có làm đăng nhập Google (mức Could) không	Chỉ làm nếu còn công suất ở sprint 6	Sprint 5

Phụ lục B. Danh sách việc tuần đầu

- ☐ Tạo repository (backend, frontend, tài liệu), bật bảo vệ nhánh main, tạo CODEOWNERS.
- ☐ Đăng ký tài khoản sandbox VNPay và lưu mã merchant, khóa bí mật ở nơi an toàn (không đưa vào Git).
- ☐ Dựng docker compose chạy SQL Server, MongoDB, MinIO, Mailpit; mọi thành viên chạy được cục bộ.
- ☐ Chốt khung OpenAPI và dựng máy chủ giả lập cho frontend.
- ☐ Spike kỹ thuật: STOMP WebSocket với MongoDB; ký HMAC-SHA512 theo VNPay; xoay vòng refresh token trong Spring Security; React-Leaflet với gom cụm marker.
- ☐ Chốt các câu hỏi OQ-02, OQ-05 và quyết định ADR còn tranh luận.
- ☐ Tạo bảng công việc từ sheet 04 và 14 cho sprint 1 và 2.
- ☐ Thống nhất quy ước ở mục 12 và cấu hình lint, định dạng, pipeline CI tối thiểu.